A

PROJECT REPORT

on

AI Powered Sanskrit NLP Framework for Morphological and Semantic Analysis

Submitted in Partial Fulfillment of the Requirements for the Course IT Project Management (21MGH303P)

Submitted In Partial Fulfillment of the Requirements
for the Degree of

Bachelor of Technology
Computer Science and Business System
of
FACULTY OF ENGINEERING AND TECHNOLOGY

by

Aditya Kumar Kapar (RA2311042030002)
Milan Kuiry (RA2311042030008)
Harsh Garg (RA2311042030010)

Under the guidance of

Dr. Anirudha Gaikwad
(Assistant Professor, Department of Computer Science & Engineering)

SRM INSTITUTE OF SCIENCE & TECHNOLOGY,
(Under Section 3 of UGC Act, 1956)
DELHI-NCR CAMPUS, MODINAGAR, U.P. (INDIA)

April, 2026

BONAFIDE CERTIFICATE

Certified that this project report titled "**AI Powered Sanskrit NLP Framework for Morphological and Semantic Analysis**" submitted for the course **IT Project Management (21MGH303P)** is the bonafide work of **Aditya Kumar Kapar (RA2311042030002), Milan Kuiry (RA2311042030008), and Harsh Garg (RA2311042030010)** who carried out the project work under my supervision during the **B.Tech. IIIrd Year / 6th Semester**. Certified further, that to the best of my knowledge, the work reported herein does not form any other project report or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

SIGNATURE

Dr. Anirudha Gaikwad
Project Supervisor & Assistant Professor
Dept. of Computer Science & Engineering

SIGNATURE 17/4/26

Dr. Rolly Gupta
Program Coordinator & Assistant Professor
Dept. of Computer Science & Engineering

SIGNATURE

Dr. Neelam Singh
Head & Associate Professor
Dept. of Computer Science & Engineering

DECLARATION

We hereby declare that the project work entitled "**AI-Powered Sanskrit NLP Framework for Morphological and Semantic Analysis**" submitted for the course **IT Project Management (21MGH303P)** to the **SRM Institute of Science & Technology, Delhi NCR Campus, Modinagar,** in partial fulfillment of the requirements for the degree of **Bachelor of Technology in Computer Science and Business System** is a record of original work carried out by us under the supervision of **Dr. Anirudha Gaikwad**, Assistant Professor, Department of Computer Science & Engineering. We further declare that this work has not been submitted to any other university or institute for the award of any degree or diploma.

ADITYA KUMAR KAPAR (RA2311042030002)
Signature: Aditya Kumar Kapar.

MILAN KUIRY (RA2311042030008)
Signature: Milan Kuiry

HARSH GARG (RA2311042030010)
Signature: Hgarg

Place: Modinagar
Date: April, 2026

ACKNOWLEDGEMENT

We take this opportunity to express our sincere gratitude and appreciation to all those who have supported us throughout the course of this project work.

We are deeply grateful to our project supervisor, **Dr. Anirudha Gaikwad**, Assistant Professor, Department of Computer Science & Engineering, SRM Institute of Science & Technology, Delhi-NCR Campus, Modinagar, for his invaluable guidance, constant encouragement, and constructive feedback that helped us complete this project successfully.

We extend our sincere thanks to **Dr. Neelam Singh**, Head & Associate Professor, Department of Computer Science & Engineering, for providing us with the necessary facilities and a conducive environment to carry out this research work.

We also express our gratitude to **Dr. Rolly Gupta**, Program Coordinator & Assistant Professor, Department of Computer Science & Engineering, for her continuous support and motivation.

We are thankful to all the faculty members and staff of the Department of Computer Science & Engineering for their assistance and cooperation.

We would also like to thank our parents and family members for their unwavering support and encouragement throughout our academic journey.

Lastly, we acknowledge the contributions of all those who directly or indirectly helped us in the successful completion of this project.

ADITYA KUMAR KAPAR (RA2311042030002)

MILAN KUIRY (RA2311042030008)

HARSH GARG (RA2311042030010)

Place: Modinagar

Date: April, 2026

Abstract

Project Title: AI-Powered Sanskrit NLP Framework for Morphological and Semantic Analysis
Course: IT Project Management (21MGH303P)
Semester: B.Tech. IIIrd Year / 6th Semester
Institution: SRM Institute of Science and Technology, Delhi-NCR Campus, Modinagar, U.P.

This report documents the comprehensive IT Project Management practical exercises conducted on the project titled *"AI-Powered Sanskrit NLP Framework for Morphological and Semantic Analysis"*. The project aims to develop an artificial intelligence-driven natural language processing framework capable of performing morphological analysis, semantic parsing, and structured knowledge extraction from classical Sanskrit texts by computationally modeling Pāṇini's Ashtadhyayi grammar.

The practical exercises are organized into five units as per the curriculum. Unit 1 covers project requirement gathering using stakeholder interviews and document analysis, project identification methodology encompassing problem identification, opportunity analysis, feasibility study (technical, economic, operational), project justification, and comprehensive stakeholder analysis with power-interest grid. Unit 2 presents software cost estimation using COCOMO (Intermediate Organic Mode), Function Point Analysis, and Expert Judgment, resulting in an estimated project cost of INR 33–41 Lakhs. Capital budgeting is performed using Net Present Value (NPV) with a positive NPV and payback period of 3–4 years. Market demand analysis identifies target segments including universities, research institutes, digital libraries, and government cultural bodies, with a three-phase demand planning strategy.

Unit 3 applies Project Evaluation and Review Technique (PERT) to compute expected durations, variance, and standard deviation for nine work breakdown structure activities, followed by Critical Path Method (CPM) analysis identifying the critical path A–B–D–E–G–H–I with a total project duration of 218 days. Resource levelling techniques are applied to resolve linguist resource constraints, and project crashing analysis determines optimal crash strategies at incremental costs. Unit 4 evaluates project performance using Earned Value Analysis (EVA) with key performance indicators including SPI, CPI, CV, and SV. Risk management identifies six key risks with quantitative risk scoring and mitigation strategies. A Risk Mitigation, Monitoring, and Management (RMMM) plan is documented, followed by configuration management

using GitHub with Git Flow branching strategy. Unit testing with 85 test cases achieves 84.7% pass rate with corrective actions for critical failures.

Unit 5 presents project audit frameworks including technical, financial, security, process, and quality audits with findings from Sprint 3. Agile frameworks are implemented using Scrum with two-week sprints, sprint planning, daily stand-ups, story boards, sprint reviews, and retrospectives. Extreme Programming (XP) practices including pair programming, test-driven development, small releases, continuous integration, refactoring, and collective code ownership are integrated with Scrum. Lean principles are applied to eliminate waste and accelerate delivery. Finally, DevOps implementation using Docker containerization is documented with Dockerfile, docker-compose, and CI/CD pipeline integration.

All cost figures are presented in Indian Rupees (INR). The analysis is conducted strictly from an IT Project Management perspective, focusing on planning, estimation, tracking, risk management, and delivery rather than software implementation details.

Total Estimated Project Cost: INR 33–41 Lakhs
Project Duration (Baseline): 218 days (approximately 10.5 months)
Payback Period: 3–4 years
Net Present Value (10% discount rate): Positive (INR 12.6 Lakhs)

Keywords: IT Project Management, Sanskrit NLP, Pāṇinian Grammar, Cost Estimation, COCOMO, Function Point Analysis, Net Present Value, PERT, CPM, Resource Levelling, Project Crashing, Earned Value Analysis, Risk Management, RMMM, Configuration Management, Agile, Scrum, Extreme Programming, DevOps, Docker.

Table of Contents

Abstract **i**

List of Tables **xii**

List of Figures **xiii**

List of Abbreviations **xiv**

1 Introduction to Project Management **1**

1.1 Experiment 1: Project Requirement Gathering and Analysis 1

1.1.1 Software Project Management and Scope 1

1.1.2 Requirement Engineering and Elicitation 1

1.1.3 Functional Requirements 2

1.1.4 Non-Functional Requirements 3

1.1.5 Requirement Prioritization using MoSCoW Method 4

1.1.6 Requirements Traceability Matrix 4

1.1.7 Project Scope Management 5

1.2 Experiment 2: Project Identification Process Methodology and Stakeholder Description 6

1.2.1 Project Identification and Selection 6

1.2.2 Feasibility Study Details 7

1.2.3 Initiation Phases and Contracting 7

1.2.4 Stakeholder Analysis 8

1.2.5 Power-Interest Grid Classification 9

1.2.6 Stakeholder Engagement Strategy 9

1.3 Experiment 3: Project Cost Estimation and Capital Budgeting 9

1.3.1 Cost Components Breakdown 9

1.3.2 Software Cost Estimation Models 10

1.3.3 Capital Budgeting: Net Present Value (NPV) 12

1.3.4 Capital Budgeting Comparison 13

1.4 Experiment 4: Market Demand Analysis and Demand Planning 13

1.4.1 Market Demand Drivers 13

1.4.2 Target Market Segments 13

1.4.3 Total Addressable Market (TAM) Estimation 14
1.4.4 Demand Planning – Three-Phase Strategy 14
1.4.5 Competitive Analysis 15
1.4.6 Market Differentiation Strategy 15
1.4.7 Risk-Adjusted Demand Forecast 15
1.4.8 Software Process Models 16
1.4.9 Conclusion for Unit 1 16

2 Project Scheduling **17**
2.1 Experiment 1: Project Evaluation and Review Technique (PERT) Analysis . . . 17
2.1.1 Work Breakdown Structure (WBS) for Sanskrit NLP Project 17
2.1.2 PERT Expected Duration Calculation 18
2.1.3 Variance and Standard Deviation 18
2.1.4 Probability Analysis 19
2.2 Experiment 2: Critical Path Method (CPM) Analysis 19
2.2.1 Precedence Relationships 19
2.2.2 Forward Pass (Earliest Start & Earliest Finish) 20
2.2.3 Backward Pass (Latest Start & Latest Finish) 20
2.2.4 Critical Path Identification 21
2.2.5 Gantt Chart Representation 21
2.3 Experiment 3: Project Resource Levelling Techniques 21
2.3.1 Project Team Formation and Structure 21
2.3.2 Selection of Project Manager 22
2.3.3 Roles and Responsibilities of Project Manager 22
2.3.4 Project Execution Strategy 23
2.3.5 Resource Requirements by Activity 23
2.3.6 Resource Availability Constraints 23
2.3.7 Resource Profile Before Levelling 23
2.3.8 Resource Levelling Strategy 24
2.3.9 Resource Profile After Levelling 24
2.3.10 Impact of Resource Levelling 25
2.4 Experiment 4: Project Crashing Techniques 25
2.4.1 Crash Analysis – Rationale 25
2.4.2 Activity Crash Options 26
2.4.3 Least Cost Crashing – Step by Step 26
2.4.4 Crash Impact on Non-Critical Paths 27
2.4.5 Crash Summary 27
2.4.6 Crash Recommendation 27
2.4.7 Time-Cost Trade-off Curve Summary 28

2.4.8 Conclusion for Unit 2 28

3 Project Monitoring **29**

3.1 Experiment 1: Project Performance Analysis 29

3.1.1 Cost, Time, Scope and Performance Monitoring 29

3.1.2 Key Performance Indicators (KPIs) 29

3.1.3 Earned Value Analysis (EVA) – Month 6 Status 30

3.1.4 Variance Analysis 30

3.1.5 Cost and Schedule Performance Indices 30

3.1.6 Forecasting 30

3.1.7 Performance Improvement Plan 30

3.1.8 Project Controlling Techniques 31

3.1.9 Project Change Control 32

3.1.10 Project Quality Controlling Techniques 32

3.1.11 Revised Forecast After Corrective Actions 33

3.2 Experiment 2: Risk Management and Mitigation 33

3.2.1 Project Risk Analysis 33

3.2.2 Risk Matrix (Probability-Impact Grid) 33

3.2.3 Risk Response Strategies 34

3.2.4 Risk Contingency Reserve 34

3.3 Experiment 3: RMMM Plan, Configuration Management & GitHub 35

3.3.1 RMMM (Risk Mitigation, Monitoring & Management) Plan 35

3.3.2 Configuration Management Plan 36

3.3.3 Software Configuration Management: GitHub 36

3.3.4 Versioning Convention 38

3.4 Experiment 4: Unit Testing with Test Cases 38

3.4.1 Testing Strategy Overview 38

3.4.2 Sample Test Case 1: Basic Noun (Nominative Singular) 38

3.4.3 Sample Test Case 2: Present Tense Verb 39

3.4.4 Sample Test Case 3: Compound Word (Sandhi) – PASS 39

3.4.5 Test Execution Summary (Sprint 4) 40

3.4.6 Defect Analysis 40

3.4.7 Corrective Actions Based on Unit Testing 40

3.4.8 Quality Metrics Dashboard 41

3.4.9 Conclusion for Unit 3 41

4 Project Closing Phase & Agile Frameworks **42**

4.1 Experiment 1: Project Audits 42

4.1.1 Project Audits: Objectives and Goals 42

4.1.2 Types of Audits . . . 42
4.1.3 Audit Checklist (Sample – Technical Audit) . . . 43
4.1.4 Audit Findings (Sprint 3) . . . 43
4.1.5 Audit Remediation Tracking . . . 44
4.1.6 Project Termination . . . 44
4.1.7 Software Testing and Testing Strategies . . . 44
4.1.8 Evaluation of Project . . . 45
4.2 Experiment 2: Agile Approaches Framework – Sprint Planning, Review . . . 46
4.2.1 Introduction to Agile Process Development . . . 46
4.2.2 Manifesto of Agile Process . . . 46
4.2.3 Agile Principles . . . 46
4.2.4 Agile Practices and Methodologies . . . 47
4.2.5 Agile Framework – Phases of Development . . . 47
4.2.6 Relationship Between Conventional and Agile . . . 47
4.2.7 IT Service Management Lifecycle . . . 48
4.2.8 Sprint Structure for Sanskrit NLP Project . . . 48
4.2.9 Sprint Planning (Sprint 4 Example) . . . 49
4.2.10 Sprint Review (Sprint 4 – Actual) . . . 49
4.3 Experiment 3: Daily Scrum Planning, Story Boards Creation, Tracking Progress 50
4.3.1 Daily Scrum (Stand-up) Format . . . 50
4.3.2 Sample Daily Scrum (Sprint 4, Day 3) . . . 50
4.3.3 Story Board Creation (Kanban Style) . . . 51
4.3.4 Tracking Progress – Sprint Burndown Chart . . . 51
4.3.5 Cumulative Flow Diagram (CFD) Metrics . . . 52
4.3.6 Sprint Retrospective – Key Action Items (Sprint 4) . . . 52
4.4 Experiment 4: Agile Approaches – Extreme Programming, Small Releases, Scrum, Lean . . . 52
4.4.1 Extreme Programming (XP) Practices Adopted . . . 52
4.4.2 Small Release Strategy . . . 53
4.4.3 Scrum Framework Integration . . . 53
4.4.4 Lean Principles Applied . . . 54
4.4.5 Scrum, XP, and Lean Comparison . . . 54
4.4.6 Benefits of Hybrid Agile Approach . . . 55
4.4.7 Conclusion for Unit 4 . . . 55

5 Scrum Methodology, DevOps & Extreme Programming 56
5.1 Experiment 1: Scrum Method and Framework . . . 56
5.1.1 Scrum Framework and Its Scope . . . 56
5.1.2 Project Management Activities in Scrum . . . 56

5.1.3 Sprint Backlog . . . 57
5.1.4 Sprint Review . . . 57
5.1.5 Retrospective Perspective . . . 58
5.1.6 Best Practices of Scrum . . . 58
5.1.7 Roles in Scrum . . . 59
5.1.8 Slack in Scrum . . . 59
5.1.9 Ten Minute Build . . . 59
5.1.10 Continuous Integration . . . 60
5.2 Experiment 2: Weekly Cycle, Pair Programming, Coding Standards . . . 60
5.2.1 Weekly Cycle Structure . . . 60
5.2.2 Pair Programming Schedule . . . 61
5.2.3 Coding Standards (Python – PEP 8 with Extensions) . . . 61
5.2.4 Code Quality Enforcement . . . 62
5.3 Experiment 3: DevOps using Docker . . . 62
5.3.1 Introduction to DevOps . . . 62
5.3.2 Docker Implementation Strategy . . . 62
5.3.3 Docker Images and Containers . . . 63
5.3.4 CI/CD Pipeline with Docker and GitHub Actions . . . 64
5.3.5 Docker Hub / GitHub Container Registry Strategy . . . 64
5.4 Experiment 4: XP Programming – Process Methodology, Framework and Limitations . . . 65
5.4.1 Introduction to Extreme Programming (XP) . . . 65
5.4.2 XP Process Methodology . . . 65
5.4.3 XP Framework . . . 65
5.4.4 XP Framework Limitations . . . 65
5.4.5 XP vs Scrum vs Lean Comparison . . . 66
5.4.6 Hybrid Approach for Sanskrit NLP Project . . . 67
5.4.7 Process Methodology Summary . . . 67
5.4.8 Continuous Integration Pipeline (Complete) . . . 67
5.4.9 Weekly Cycle Effectiveness Metrics . . . 69
5.4.10 Pair Programming Benefits Quantified . . . 69
5.4.11 Conclusion for Unit 5 . . . 69

References **72**

Appendices **73**
Appendix A: Detailed WBS . . . 73
Appendix B: Risk Register . . . 74
Appendix C: Complete Test Cases . . . 75

Appendix D: Sprint Burndown Data 76
Appendix E: Project Charter 77
Appendix F: Stakeholder Communication Plan 78
Appendix G: Quality Assurance Checklist 78
Appendix H: Tools and Technologies 78
Appendix I: Lessons Learned 79
Appendix J: Acronyms and Glossary 80
Appendix K: Budget Breakdown 80
Appendix L: Resource Loading Chart 80

List of Tables

1.1 Functional Requirements for Sanskrit NLP Framework 2
1.2 Non-Functional Requirements 3
1.3 Requirements Traceability Matrix 4
1.4 Project Identification Phases 6
1.5 Stakeholder Identification and Analysis 8
1.6 Power-Interest Grid (Adapted from Freeman [1]) 9
1.7 Stakeholder Engagement Plan 9
1.8 Cost Components Breakdown 10
1.9 COCOMO Input Parameters 10
1.10 Function Point Counts 11
1.11 NPV Assumptions 12
1.12 Capital Budgeting Techniques Comparison 13
1.13 Target Market Segments 13
1.14 TAM Estimation by Year 14
1.15 Three-Phase Demand Planning Strategy 14
1.16 Competitive Analysis 15
1.17 Risk-Adjusted Demand Forecast 15

2.1 Work Breakdown Structure with Time Estimates (in days) 17
2.2 PERT Expected Duration Calculation 18
2.3 Variance and Standard Deviation by Activity 19
2.4 Activity Precedence Relationships 19
2.5 Forward Pass Calculation 20
2.6 Backward Pass and Slack Calculation 20
2.7 Gantt Chart Summary (Activities by Time Period) 21
2.8 Project Team Structure 22
2.9 Project Manager Roles and Responsibilities 22
2.10 Resource Requirements by Activity 23
2.11 Resource Profile Before Levelling 24
2.12 Resource Profile After Levelling 24
2.13 Resource Levelling Impact Summary 25
2.14 Activity Crash Options 26

2.15 Least Cost Crashing Steps . . . 26
2.16 Non-Critical Path Impact . . . 27
2.17 Crash Summary . . . 27
2.18 Time-Cost Trade-off Summary . . . 28

3.1 Key Performance Indicators . . . 29
3.2 Earned Value Analysis Parameters . . . 30
3.3 Performance Improvement Plan . . . 31
3.4 Change Control Process . . . 32
3.5 Quality Control Techniques . . . 32
3.6 Revised Forecast . . . 33
3.7 Risk Identification and Assessment . . . 33
3.8 Risk Response Strategies . . . 34
3.9 Risk Contingency Reserve . . . 34
3.10 RMMM Monitoring Schedule . . . 35
3.11 Risk Triggers and Thresholds . . . 35
3.12 Git Flow Branching Strategy . . . 37
3.13 CI/CD Pipeline Stages . . . 37
3.14 Testing Strategy . . . 38
3.15 Test Case TC-MORPH-001 . . . 38
3.16 Test Case TC-MORPH-015 . . . 39
3.17 Test Case TC-MORPH-042 (Passing) . . . 39
3.18 Test Execution Summary . . . 40
3.19 Defect Analysis by Severity . . . 40
3.20 Quality Metrics Dashboard . . . 41

4.1 Types of Audits for Sanskrit NLP Project . . . 42
4.2 Audit Findings and Corrective Actions . . . 43
4.3 Audit Remediation Tracking . . . 44
4.4 Types of Testing for Sanskrit NLP Project . . . 45
4.5 Agile Principles and Their Application . . . 46
4.6 Agile Development Phases . . . 47
4.7 Conventional vs Agile Project Management . . . 47
4.8 Sprint Structure . . . 48
4.9 Sprint Planning Example . . . 49
4.10 Sprint Review Results . . . 49
4.11 Daily Scrum Example . . . 50
4.12 Story Board (Kanban) . . . 51
4.13 Sprint Burndown Data (Sprint 4) . . . 51
4.14 Cumulative Flow Diagram Data . . . 52

4.15 XP Practices for Sanskrit NLP Project 52
4.16 Small Release Plan 53
4.17 Scrum + XP Integration 54
4.18 Lean Principles in Sanskrit NLP Project 54
4.19 Comparison of Agile Methodologies 55

5.1 Scrum Project Management Activities 56
5.2 Sprint Backlog Example (Sprint 5) 57
5.3 Sprint Review Structure 57
5.4 Sprint Retrospective Structure 58
5.5 Scrum Best Practices for Sanskrit NLP Project 58
5.6 Scrum Roles and Responsibilities 59
5.7 Slack Allocation in Sprints 59
5.8 Ten Minute Build Implementation 59
5.9 CI Pipeline Implementation 60
5.10 Weekly Agile Cycle 60
5.11 Pair Programming Schedule 61
5.12 Coding Standards 61
5.13 Docker Images for Sanskrit NLP Project 63
5.14 CI/CD Pipeline Stages 64
5.15 XP Practices for Sanskrit NLP Project 65
5.16 XP Limitations and Mitigation Strategies 65
5.17 Comparison of Agile Methodologies Used 66
5.18 Process Methodology Summary 67
5.19 Weekly Cycle Effectiveness Over Time 69
5.20 Pair Programming Benefits 69
5.21 Final Project Summary 70
5.22 Detailed Work Breakdown Structure 73
5.23 Complete Risk Register 74
5.24 Noun Declension Test Cases 75
5.25 Verb Conjugation Test Cases 75
5.26 Sandhi Splitting Test Cases 76
5.27 Test Execution Log 76
5.28 Sprint Performance Summary 76
5.29 Sprint Burndown Detailed Data 77
5.30 Project Charter 77
5.31 Stakeholder Communication Plan 78
5.32 Quality Assurance Checklist 78
5.33 Tools and Technologies 78

5.34 Glossary of Terms 80
5.35 Budget Breakdown by Phase (INR Lakhs) 80
5.36 Resource Loading by Month 81

List of Figures

5.1 XP Process Flow for Sanskrit NLP Project 68

List of Abbreviations

Abbreviation	Full Form
AI	Artificial Intelligence
NLP	Natural Language Processing
ML	Machine Learning
PERT	Project Evaluation and Review Technique
CPM	Critical Path Method
EVA	Earned Value Analysis
EV	Earned Value
PV	Planned Value
AC	Actual Cost
CPI	Cost Performance Index
SPI	Schedule Performance Index
BAC	Budget at Completion
EAC	Estimate at Completion
RMMM	Risk Mitigation, Monitoring and Management
CCB	Change Control Board
CI/CD	Continuous Integration / Continuous Deployment
XP	Extreme Programming
TDD	Test-Driven Development
UAT	User Acceptance Testing
KLOC	Thousand Lines of Code
FP	Function Points
NPV	Net Present Value
IRR	Internal Rate of Return
UGC	University Grants Commission
IGNCA	Indira Gandhi National Centre for the Arts
API	Application Programming Interface

Unit 1

Introduction to Project Management

1.1 Experiment 1: Project Requirement Gathering and Analysis

1.1.1 Software Project Management and Scope

Software project management encompasses the planning, organizing, monitoring, and controlling of software development projects [2]. The AI-Powered Sanskrit NLP Framework project scope includes the development of an NLP system capable of morphological analysis, semantic parsing, and knowledge extraction from classical Sanskrit texts using Paninian grammar.

1.1.2 Requirement Engineering and Elicitation

Requirement engineering involves the systematic process of eliciting, analyzing, documenting, and validating requirements [3]. The requirement gathering process for this project employed a hybrid approach combining three primary techniques.

Primary Technique: Facilitated Workshops

Three facilitated workshops were conducted with a duration of 2–3 hours each, involving:

- Two Sanskrit linguists specializing in Paninian grammar from Banaras Hindu University
- Two NLP researchers with experience in low-resource language processing
- The project manager and two senior NLP engineers

Secondary Technique: Document Analysis

The following documents were analyzed:

- Ashtadhyayi (Panini's grammar) – selected sutras relevant to morphological analysis [4]

- Existing Sanskrit NLP tools documentation (Sanskrit Heritage Engine [5], DCS, SanskritTagger)
- Project charter and problem statement

Tertiary Technique: Survey

A structured survey was distributed to 15 academic institutions across India, USA, and Europe working in digital humanities and computational linguistics. Response rate: 73% (11 responses).

1.1.3 Functional Requirements

Table 1.1: Functional Requirements for Sanskrit NLP Framework

ID	Requirement Description	Source	Priority
FR01	Morphological analysis of Sanskrit words using Paninian rules (Ashtadhyayi) [4]	Workshop	High
FR02	Semantic parsing of classical Sanskrit verses (subanta and tinanta structures)	Workshop	High
FR03	Knowledge extraction in subject-predicate-object triple format	Survey	High
FR04	REST API for researchers to submit Sanskrit text programmatically	Survey	Medium
FR05	Export parsed grammar trees as XML and JSON formats	Document analysis	Medium
FR06	Batch processing capability for large Sanskrit corpora (e.g., Mahabharata)	Workshop	High
FR07	Bilingual interface (Devanagari + IAST transliteration)	Survey	Medium

FR08	User authentication and role-based access for academic users	Document analysis	Low
FR09	Versioning of grammar rule updates for reproducibility	Workshop	Medium
FR10	Integration with existing Sanskrit digital libraries (GRETIL, SARIT)	Survey	Medium

1.1.4 Non-Functional Requirements

Table 1.2: Non-Functional Requirements

Category	**Requirement**	**Target Metric**	**Priority**
Performance	Morphological analysis latency	< 2 seconds per verse (up to 100 words)	High
Accuracy	Precision on test dataset	$> 92\%$ validated by Sanskrit experts	High
Scalability	Maximum corpus size	10 million word tokens	Medium
Availability	API uptime for academic use	99.5%	Medium
Security	Data protection and access control	Role-based access; no public modification	Low
Maintainability	Modular design for rule updates	New sutra addition within 2 person-days	High
Usability	Learning curve for researchers	< 4 hours for basic API usage	Medium
Reliability	Mean Time Between Failures (MTBF)	> 200 hours of continuous operation	Medium

1.1.5 Requirement Prioritization using MoSCoW Method

The MoSCoW prioritization framework [6] was applied to classify requirements:

Must Have (Critical for v1.0)

- Morphological analyzer based on Paninian rules (FR01)
- Panini rule engine with support for minimum 500 sutras
- Semantic parser for basic verse structures (FR02)
- Accuracy validation framework (NFR-Accuracy)

Should Have (Important but not blocking v1.0)

- Knowledge extraction module (FR03)
- REST API with basic documentation (FR04)
- Batch processing for corpora up to 1 million words (FR06)

Could Have (Desirable for v2.0)

- Visual syntax tree visualization
- Bilingual interface (FR07)
- Integration with external digital libraries (FR10)

Won't Have (Out of scope for this project)

- Speech synthesis or recognition for Sanskrit
- Machine translation from Sanskrit to English or Hindi
- Optical character recognition (OCR) for handwritten manuscripts

1.1.6 Requirements Traceability Matrix

Table 1.3: Requirements Traceability Matrix

Req ID	Source	Business Driver	Validation Method

FR01	Workshop (Linguists)	Digital preservation of Paninian grammar [4]	Unit test with 500 annotated Sanskrit words
FR02	Workshop (Linguists)	Semantic understanding for research	Expert review of 100 semantic frames
FR03	Survey (Researchers)	Structured knowledge extraction	Sample extraction from Ramayana corpus
FR04	Survey (Researchers)	Accessibility for external users	API endpoint testing with Postman
NFR-Perf	Feasibility study	Acceptable user experience	Load test with 50 concurrent requests
NFR-Acc	Workshop	Scholarly credibility	Cross-validation with 3 Sanskrit experts

1.1.7 Project Scope Management

In Scope

1. Development of morphological analyzer covering minimum 500 Panini sutras
2. Semantic parser for classical Sanskrit texts (pre-1000 CE)
3. Knowledge extraction module outputting structured triples
4. REST API with Swagger/OpenAPI documentation
5. Deployment on cloud infrastructure (AWS/GCP)
6. Sample Jupyter notebooks for researchers
7. User documentation and API reference

Out of Scope

1. Mobile application development
2. Real-time processing of streaming text
3. Support for post-1000 CE Sanskrit (Kavya, modern Sanskrit)
4. Integration with proprietary commercial systems

5. Training data annotation beyond project corpus

Acceptance Criteria

1. Morphological analysis accuracy $\geq 92\%$ on gold-standard test set [7]
2. API response time ≤ 2 seconds for 100-word input
3. Successful batch processing of Mahabharata (approximately 1.8 million words)
4. Positive validation from at least 2 external Sanskrit linguists

1.2 Experiment 2: Project Identification Process Methodology and Stakeholder Description

1.2.1 Project Identification and Selection

The project identification followed a structured five-phase methodology as documented in the PMBOK Guide [2] and Kerzner's project management framework [8]:

Table 1.4: Project Identification Phases

Phase	**Activity**	**Outcome for Sanskrit NLP Project**
Problem Identification	Systematic analysis of gaps in existing Sanskrit NLP tools [9]	Identified lack of integrated Paninian grammar engine with modern ML
Opportunity Analysis	Assessment of AI and NLP advancements for classical language processing	AI transformer models can be adapted for Sanskrit morphology
Feasibility Study	Technical, economic, and operational feasibility assessment [3]	Technical: Feasible with rule-ML hybrid; Economic: INR 33-41 Lakhs; Operational: High scholarly demand
Project Justification	Alignment with cultural, academic, and research objectives	Cultural preservation, linguistic research, AI for low-resource languages

Approval Gate	Submission to funding and oversight bodies	Approved by SRM, IIT (BHU) Sanskrit Department, and National Mission for Manuscripts

1.2.2 Feasibility Study Details

Technical Feasibility

- Existing Panini rule formalisms can be extended [5]
- Hybrid approach: rule-based morphological analysis combined with ML-based disambiguation [7]
- Technology stack: Python, spaCy, PyTorch, FastAPI
- **Identified Risk**: Complexity of sandhi (phonetic merging) rules requires expert linguist involvement

Economic Feasibility

- Total estimated cost: INR 33–41 Lakhs (using COCOMO model [10])
- Expected benefits: Academic licenses (INR 1.5 Lakhs per year per institution) and research grants
- Net Present Value (10% discount rate): Positive (INR 12.6 Lakhs over 5 years)
- Payback period: 3–4 years

Operational Feasibility

- Linguist availability: 3 senior Sanskrit experts willing to collaborate
- NLP engineering team: 6 engineers can be recruited within 2 months
- Infrastructure: Cloud services readily available
- **Constraint**: Limited number of Sanskrit NLP experts globally

1.2.3 Initiation Phases and Contracting

The project initiation involves the following phases:

1. **Project Charter Development**: Formal authorization document signed by sponsor

2. **Stakeholder Identification**: Mapping all parties with interest in project outcomes
3. **Contracting**: Memorandum of Understanding between SRM, IIT (BHU), and National Mission for Manuscripts

1.2.4 Stakeholder Analysis

Stakeholder analysis follows the framework proposed by Freeman [1] and Clegg et al. [11].

Table 1.5: Stakeholder Identification and Analysis

Stakeholder	**Role in Project**	**Interest Level**	**Power or Influence**	**Expectation**
UGC and MHRD (Sponsor)	Funding approval and oversight	High	High	On-time delivery with scholarly impact
Project Manager	Planning, execution, and reporting	High	Medium	Budget and schedule adherence
Sanskrit Linguists (3)	Grammar modeling and validation	Very High	High	Accurate Panini implementation [4]
NLP Engineers (4)	Model development and API	High	Medium	Clear requirements with modern stack
Data Scientists (2)	Corpus training and annotation	Medium	Medium	Quality annotated data
IGNCA (Cultural body)	Preservation partner	Medium	Medium	Usable digital archive
University Researchers	End users	High	Low	Accurate and easy-to-use API
Graduate Students	End users	Medium	Low	Learning resource and documentation

Cloud Provider (AWS)	Infrastructure hosting	Low	Low (critical)	Service level compliance

1.2.5 Power-Interest Grid Classification

Table 1.6: Power-Interest Grid (Adapted from Freeman [1])

Quadrant	**Stakeholders**	**Engagement Strategy**
High Power, High Interest	Project Sponsor, Sanskrit Linguists	Manage Closely – Monthly meetings, weekly reviews
High Power, Low Interest	IGNCA, University Administration	Keep Satisfied – Quarterly reports
Low Power, High Interest	Researchers, Graduate Students	Keep Informed – Monthly newsletter, beta access
Low Power, Low Interest	Cloud Provider	Monitor – Automated alerts

1.2.6 Stakeholder Engagement Strategy

Table 1.7: Stakeholder Engagement Plan

Stakeholder	**Engagement Method**	**Frequency**
UGC and MHRD	Formal status reports, steering committee	Monthly
Sanskrit Linguists	Joint review sessions, pair validation	Weekly
NLP Engineers	Daily stand-ups, sprint planning	Daily
Researchers	User feedback surveys, beta testing	Per release
IGNCA	Quarterly review meeting	Quarterly
Graduate Students	Documentation review, usability testing	Per milestone

1.3 Experiment 3: Project Cost Estimation and Capital Budgeting

1.3.1 Cost Components Breakdown

Table 1.8: Cost Components Breakdown

Cost Category	Description	Estimated Range (INR)
Development	NLP model development, Panini grammar engine, API development	18–22 Lakhs
Data Annotation	Corpus collection, expert annotation, validation	6–8 Lakhs
Infrastructure	Cloud compute (AWS/GCP), storage, bandwidth	3–4 Lakhs
Tools & Licenses	NLP frameworks, ML libraries, IDEs	0.5–1 Lakhs
Testing & QA	Accuracy validation, performance testing	2 Lakhs
Project Management	PM tools, collaboration software	1 Lakhs
Contingency (15%)	Unforeseen expenses, risk mitigation	4–5 Lakhs
Total		**33–41 Lakhs**

1.3.2 Software Cost Estimation Models

COCOMO (Intermediate Organic Mode)

The Constructive Cost Model (COCOMO) Intermediate model [10] was applied:

Table 1.9: COCOMO Input Parameters

Parameter	Value
Project Type	Organic (relatively small team, familiar environment, but complex domain)
Estimated Project Size	25–30 KLOC (thousand lines of code)
Effort Formula	$E = a \times (KLOC)^b \times \prod_{i=1}^{n}(EM_i)$
Organic Mode Constants	$a = 3.2$, $b = 1.05$

Effort Multipliers (Selected):

- Required software reliability: High (1.15)
- Product complexity: Very High (1.30) – due to Panini rule complexity
- Analyst capability: High (0.86) – senior NLP experts required
- Programmer capability: High (0.86)
- Application experience: Low (1.13) – niche Sanskrit NLP domain
- Use of software tools: High (0.91)

- Required development schedule: Normal (1.00)

Calculated Effort:

$$E = 3.2 \times (27.5)^{1.05} \times (1.15 \times 1.30 \times 0.86 \times 0.86 \times 1.13 \times 0.91)$$

$$E \approx 3.2 \times 33.2 \times 1.12 \approx 119 \text{ Person-Months}$$

Adjusted Effort: 70–90 Person-Months

Function Point Analysis

Function Point Analysis [3] was performed:

Table 1.10: Function Point Counts

Function Type	**Count**	**Complexity Weight**
External Inputs (Sanskrit text submission)	3	High (6)
External Outputs (Parsed grammar trees)	4	High (7)
External Inquiries (API queries)	3	Medium (4)
Internal Logical Files (Panini rule database)	2	High (15)
External Interface Files (Corpus imports)	2	Medium (7)

Unadjusted Function Points (UFP) $= (3 \times 6) + (4 \times 7) + (3 \times 4) + (2 \times 15) + (2 \times 7) = 102$

Value Adjustment Factor (VAF) – Total Degree of Influence: 40

$$\text{VAF} = 0.65 + (40/100) = 1.05$$

$$\text{Adjusted Function Points} = 102 \times 1.05 = 107 \text{ FP}$$

Considering high complexity of Panini grammar, **estimated FP of 450–500** was adopted.

Expert Judgment

Expert judgment from NLP engineers and Sanskrit researchers yielded:

- Sanskrit NLP niche domain → specialist cost premium of 30–40%
- Rule-engine development inflates effort by 25%
- Data annotation requires linguist involvement → INR 8–12 per word

Consensus Estimate: INR 33–41 Lakhs

1.3.3 Capital Budgeting: Net Present Value (NPV)

Assumptions

Table 1.11: NPV Assumptions

Parameter	Value
Discount rate (cost of capital)	10% per annum
Project life	5 years (after development)
Initial investment (Year 0)	INR 37 Lakhs (midpoint)
Annual operating costs	INR 6 Lakhs

Annual Benefits:

- Academic licenses (10 institutions @ INR 1.5 Lakhs) = INR 15 Lakhs
- Research grants = INR 5 Lakhs
- API subscriptions = INR 3 Lakhs
- **Total**: INR 23 Lakhs (Year 1-2) → INR 28 Lakhs (Year 3-5)

Net annual cash flow: INR 17 Lakhs (Year 1-2), INR 22 Lakhs (Year 3-5)

NPV Calculation

$$NPV = -37 + \frac{17}{1.10} + \frac{17}{(1.10)^2} + \frac{22}{(1.10)^3} + \frac{22}{(1.10)^4} + \frac{22}{(1.10)^5}$$

$$NPV = -37 + 15.45 + 14.05 + 16.53 + 15.03 + 13.66 = 37.72 \text{ Lakhs}$$

Payback Period

Cumulative cash flows:

- Year 0: -37 Lakhs
- Year 1: -20 Lakhs

- Year 2: -3 Lakhs
- Year 3: +19 Lakhs

$$\text{Payback Period} = 2 + \frac{3}{22} = 2.14 \text{ years}$$

1.3.4 Capital Budgeting Comparison

Table 1.12: Capital Budgeting Techniques Comparison

Technique	Result	Decision
Net Present Value (NPV)	INR +37.72 Lakhs	Accept (positive)
Payback Period	2.14 years	Accept ($<$ 5 years)
Internal Rate of Return (IRR)	$\approx 42\%$	Accept ($>$ 10%)
Profitability Index (PI)	$74.72/37 = 2.02$	Accept ($>$ 1)

1.4 Experiment 4: Market Demand Analysis and Demand Planning

1.4.1 Market Demand Drivers

1. **Growing Interest in Digital Humanities**: Global market USD 4.2 billion (2025), CAGR 14.3%
2. **Government-Funded Language Preservation**: National Mission for Manuscripts – 5 million manuscripts [12]
3. **EdTech Platforms**: Sanskrit enrollment grew 40% YoY (2023-2025)
4. **UNESCO Cultural Digitization**: Memory of the World Programme
5. **AI Research in Low-Resource Languages**: Funding from DARPA, European Research Council

1.4.2 Target Market Segments

Table 1.13: Target Market Segments

Segment	Demand Level	Estimated Market Size

Universities & Research Institutes	High	120 (India) + 80 (International)
Digital Libraries	Medium	40 major collections globally
Language Learning Platforms	Medium	15–20 platforms
Government Cultural Bodies	High	8–10 (NMM, IGNCA, Sanskruti)
AI Research Labs	Medium	25–30 labs

1.4.3 Total Addressable Market (TAM) Estimation

Table 1.14: TAM Estimation by Year

Year	**Target**	**Revenue (INR)**	**Market Penetration**
Year 1	5 universities (free pilot)	0	Pilot phase
Year 2	15 paid institutions @ INR 1.5 Lakhs	22.5 Lakhs	7.5%
Year 3	35 institutions + 5 API subscribers	57.5 Lakhs	17.5%
Year 4	50 institutions + 10 API subscribers	85 Lakhs	25%
Year 5	60 institutions + 15 API subscribers + 3 libraries	114 Lakhs	30%

1.4.4 Demand Planning – Three-Phase Strategy

Table 1.15: Three-Phase Demand Planning Strategy

Phase	**Timeline**	**Activities & Targets**
Phase 1: Academic Adoption	Year 1	Free pilot to 5 leading Sanskrit universities; Collect feedback; Aim: 90% satisfaction
Phase 2: API & Research Tools	Years 2-3	Paid academic licenses (INR 1.5 Lakhs/year); Target: 35 institutions by Year 3

Phase 3: EdTech & Multilingual Expansion	Years 4-5	Integrate with 3 EdTech platforms; Expand to Pali, Prakrit; Aim: INR 114 Lakhs revenue

1.4.5 Competitive Analysis

Table 1.16: Competitive Analysis

Competitor	**Strengths**	**Weaknesses**	**Threat Level**
Sanskrit Heritage Engine (CNRS) [5]	Well-established, rule-based	Limited ML, complex setup	Medium
DCS (Cambridge) [13]	Good corpus search	No semantic parsing	Low
SanskritTagger (Open source)	Free, simple	Low accuracy (<70%)	Low
Manual linguistic analysis	High accuracy	Not scalable	Low

1.4.6 Market Differentiation Strategy

1. **Hybrid Approach**: Rule-based Paninian grammar + ML disambiguation [7]
2. **Semantic Parsing**: Extracts meaning, not just morphological tags
3. **API-First Design**: Easy integration for researchers
4. **Structured Knowledge Output**: Machine-readable triples
5. **Active Linguist Involvement**: Continuous expert validation

1.4.7 Risk-Adjusted Demand Forecast

Table 1.17: Risk-Adjusted Demand Forecast

Scenario	**Probability**	**Year 3 Revenue**	**Year 5 Revenue**	**NPV Impact**
Optimistic	25%	INR 80 Lakhs	INR 150 Lakhs	+55 Lakhs
Most Likely	50%	INR 57.5 Lakhs	INR 114 Lakhs	+37.7 Lakhs
Pessimistic	25%	INR 35 Lakhs	INR 70 Lakhs	+15 Lakhs

1.4.8 Software Process Models

The project follows a hybrid software process model combining:

- **Traditional Waterfall Model**: For Panini rule formalization (sequential, well-defined)
- **Incremental Model**: For morphological analyzer (releases v1, v2, v3)
- **Agile Methodology**: For API development and testing (2-week sprints)

1.4.9 Conclusion for Unit 1

The requirement gathering phase successfully identified 10 functional requirements and 8 non-functional requirements, prioritized using the MoSCoW method [6]. The project identification methodology confirmed technical, economic, and operational feasibility with an estimated budget of INR 33-41 Lakhs, following COCOMO and Function Point Analysis [3,10]. Stakeholder analysis [1] identified 9 key stakeholders with tailored engagement strategies. Market demand analysis projects INR 114 Lakhs in annual revenue by Year 5 with a three-phase adoption strategy.

Unit 2

Project Scheduling

2.1 Experiment 1: Project Evaluation and Review Technique (PERT) Analysis

2.1.1 Work Breakdown Structure (WBS) for Sanskrit NLP Project

The Work Breakdown Structure (WBS) is a hierarchical decomposition of the project into manageable activities [2]. The Sanskrit NLP Framework project is divided into nine major activities:

Table 2.1: Work Breakdown Structure with Time Estimates (in days)

Task ID	Activity Description	Optimistic (O)	Pessimistic (P)	Most Likely (M)
A	Requirement gathering & stakeholder sign-off	10	20	14
B	Corpus collection, digitization & expert annotation	30	50	38
C	Panini rule formalization (Ashtadhyayi sutras)	40	70	52
D	Morphological analyzer development	35	60	45
E	Semantic parser development	30	55	40
F	Knowledge extraction module development	20	40	28
G	API development & documentation	15	30	20

H	Integration testing & accuracy validation	20	40	28
I	Deployment, user training & project closure	10	20	14

2.1.2 PERT Expected Duration Calculation

The PERT expected duration formula [2] is:

$$T_e = \frac{O + 4M + P}{6}$$

Where:

- O = Optimistic time estimate
- M = Most likely time estimate
- P = Pessimistic time estimate

Table 2.2: PERT Expected Duration Calculation

Activity	**Calculation**	**Expected Duration (days)**
A	$(10 + 4 \times 14 + 20)/6$	14.33
B	$(30 + 4 \times 38 + 50)/6$	38.67
C	$(40 + 4 \times 52 + 70)/6$	53.00
D	$(35 + 4 \times 45 + 60)/6$	45.83
E	$(30 + 4 \times 40 + 55)/6$	40.83
F	$(20 + 4 \times 28 + 40)/6$	28.67
G	$(15 + 4 \times 20 + 30)/6$	20.83
H	$(20 + 4 \times 28 + 40)/6$	28.67
I	$(10 + 4 \times 14 + 20)/6$	14.33

2.1.3 Variance and Standard Deviation

Variance formula:

$$\sigma^2 = \left(\frac{P - O}{6}\right)^2$$

Standard deviation:

$$\sigma = \sqrt{\sigma^2}$$

Table 2.3: Variance and Standard Deviation by Activity

Activity	**Calculation**	**Variance** (σ^2)	**Std Dev** (σ)
A	$((20-10)/6)^2$	2.78	1.67
B	$((50-30)/6)^2$	11.11	3.33
C	$((70-40)/6)^2$	25.00	5.00
D	$((60-35)/6)^2$	17.36	4.17
E	$((55-30)/6)^2$	17.36	4.17
F	$((40-20)/6)^2$	11.11	3.33
G	$((30-15)/6)^2$	6.25	2.50
H	$((40-20)/6)^2$	11.11	3.33
I	$((20-10)/6)^2$	2.78	1.67

2.1.4 Probability Analysis

Total variance along critical path (identified in CPM analysis):

$$\sigma^2_{total} = \sigma^2_A + \sigma^2_C + \sigma^2_D + \sigma^2_E + \sigma^2_G + \sigma^2_H + \sigma^2_I$$

$$\sigma^2_{total} = 2.78 + 25.00 + 17.36 + 17.36 + 6.25 + 11.11 + 2.78 = 82.64$$

$$\sigma_{total} = \sqrt{82.64} = 9.09 \text{ days}$$

Probability of completing the project within 230 days:

$$Z = \frac{230 - 218}{9.09} = 1.32$$

From standard normal distribution table: $P(Z \leq 1.32) \approx 0.9066$ or 90.66%

2.2 Experiment 2: Critical Path Method (CPM) Analysis

2.2.1 Precedence Relationships

Table 2.4: Activity Precedence Relationships

Activity	**Predecessor(s)**	**Rationale**
A	-	Starting activity – requirements must be gathered first
B	A	Corpus selection depends on requirements

C	A	Panini rule formalization depends on requirements
D	B, C	Morphological analyzer requires both corpus and rules
E	D	Semantic parser depends on morphological analyzer
F	D	Knowledge extraction depends on morphological analyzer
G	E, F	API requires both semantic parser and knowledge extraction
H	G	Testing requires completed API
I	H	Deployment requires successful testing

2.2.2 Forward Pass (Earliest Start & Earliest Finish)

Assuming project start at day 0:

Table 2.5: Forward Pass Calculation

Activity	**Earliest Start (ES)**	**Earliest Finish (EF)**
A	0	14.33
B	14.33	53.00
C	14.33	67.33
D	67.33	113.16
E	113.16	154.00
F	113.16	141.83
G	154.00	174.83
H	174.83	203.50
I	203.50	217.83

2.2.3 Backward Pass (Latest Start & Latest Finish)

Table 2.6: Backward Pass and Slack Calculation

Activity	**Latest Start (LS)**	**Latest Finish (LF)**	**Slack (LS - ES)**
A	0	14.33	0
B	28.67	67.34	14.34
C	14.34	67.34	0.01

D	67.34	113.17	0.01
E	113.17	154.00	0.01
F	125.33	154.00	12.17
G	154.00	174.83	0
H	174.83	203.50	0
I	203.50	217.83	0

2.2.4 Critical Path Identification

Activities with zero slack constitute the critical path:

$$\mathbf{A} \rightarrow \mathbf{C} \rightarrow \mathbf{D} \rightarrow \mathbf{E} \rightarrow \mathbf{G} \rightarrow \mathbf{H} \rightarrow \mathbf{I}$$

Total Project Duration: 217.83 days (approximately 218 days or 10.5 months)

Note: Activity B (corpus annotation) has 14.34 days of slack and can be delayed without affecting the project completion date. Activity F (knowledge extraction) has 12.17 days of slack.

2.2.5 Gantt Chart Representation

Table 2.7: Gantt Chart Summary (Activities by Time Period)

Activity	**Duration**	**Timeline (Days)**
A	14 days	Days 0-14
B	39 days	Days 14-53
C	53 days	Days 14-67
D	46 days	Days 67-113
E	41 days	Days 113-154
F	29 days	Days 113-142
G	21 days	Days 154-175
H	29 days	Days 175-204
I	14 days	Days 204-218

2.3 Experiment 3: Project Resource Levelling Techniques

2.3.1 Project Team Formation and Structure

The project team consists of the following roles:

Table 2.8: Project Team Structure

Role	Number	Key Responsibilities
Project Manager	1	Overall planning, execution, stakeholder communication
Lead Sanskrit Linguist	1	Grammar validation, rule accuracy
Sanskrit Linguists	2	Annotation, rule formalization
Lead NLP Engineer	1	Architecture design, technical decisions
NLP Engineers	3	Development, testing, documentation
Data Scientist	1	Corpus processing, model training
QA Engineer	1	Testing, quality assurance

2.3.2 Selection of Project Manager

The Project Manager was selected based on the following criteria:

- Minimum 5 years of experience in software project management
- Prior experience in NLP or AI projects
- PMP certification preferred
- Excellent stakeholder management skills
- Experience with academic and government-funded projects

2.3.3 Roles and Responsibilities of Project Manager

Table 2.9: Project Manager Roles and Responsibilities

Area	Responsibilities
Planning	Develop project plan, WBS, schedule, and budget
Execution	Coordinate team activities, manage resources
Monitoring	Track progress, EVM analysis, risk monitoring
Communication	Stakeholder reporting, steering committee meetings
Quality	Ensure deliverables meet acceptance criteria
Closure	Final reporting, knowledge transfer, lessons learned

2.3.4 Project Execution Strategy

Project execution follows a phased approach:

1. **Phase 1 (Days 0-14)**: Requirement finalization and team onboarding
2. **Phase 2 (Days 14-67)**: Corpus collection and Panini rule formalization
3. **Phase 3 (Days 67-154)**: Core NLP module development
4. **Phase 4 (Days 154-204)**: API development and testing
5. **Phase 5 (Days 204-218)**: Deployment and closure

2.3.5 Resource Requirements by Activity

The project requires two types of critical resources: NLP Engineers and Sanskrit Linguists.

Table 2.10: Resource Requirements by Activity

Activity	Duration (days)	NLP Engineers	Linguists	Total Resources
A	14.33	1	2	3
B	38.67	2	3	5
C	53.00	2	4	6
D	45.83	3	2	5
E	40.83	3	2	5
F	28.67	2	1	3
G	20.83	2	0	2
H	28.67	2	2	4
I	14.33	1	1	2

2.3.6 Resource Availability Constraints

- **NLP Engineers available**: 4 (maximum)
- **Sanskrit Linguists available**: 4 (maximum – constraint based on expert availability)
- **Current peak NLP requirement**: 5 engineers (during activities D and E overlapping)
- **Current peak Linguist requirement**: 6 linguists (during activity C)

2.3.7 Resource Profile Before Levelling

Table 2.11: Resource Profile Before Levelling

Time Period (days)	**Activities**	**NLP Required**	**Linguists Required**	**Constraint Violation**
0 – 14	A	1	2	None
14 – 53	B, C	4 (2+2)	7 (3+4)	Linguists ($7 > 4$)
53 – 67	C	2	4	None (at limit)
67 – 113	D, E, F	8 (3+3+2)	5 (2+2+1)	NLP ($8 > 4$); Linguists ($5 > 4$)
113 – 141	E, F	5 (3+2)	3 (2+1)	NLP ($5 > 4$)
141 – 154	E	3	2	None
154 – 175	G	2	0	None
175 – 203	H	2	2	None
203 – 217	I	1	1	None

2.3.8 Resource Levelling Strategy

To resolve resource over-allocation, the following strategies are applied:

Strategy 1 – Shift Non-Critical Activities:

- Activity B (slack = 14.34 days) – shift start by 10 days to reduce overlap with C
- Activity F (slack = 12.17 days) – shift start by 15 days to reduce overlap with D and E

Strategy 2 – Split Critical Activities:

- Activity C (Panini rule formalization) – split into two sub-phases:
  - C1: 30 days with 4 linguists
  - C2: 23 days with 2 linguists

Strategy 3 – Add Additional Resource (if budget permits):

- Hire 1 additional linguist on contract (INR 1.5 Lakhs for 3 months)
- Hire 1 additional NLP engineer (INR 2 Lakhs for 4 months)

2.3.9 Resource Profile After Levelling

Table 2.12: Resource Profile After Levelling

Time Period (days)	NLP Required	Linguists Required	Status
0 – 14	1	2	OK
14 – 44	4	4	OK (after shifting B and splitting C)
44 – 67	3	3	OK
67 – 98	4	3	OK
98 – 113	5	3	OK (NLP at limit)
113 – 141	4	2	OK
141 – 154	3	2	OK
154 – 175	2	0	OK
175 – 203	2	2	OK
203 – 232	1	1	OK

2.3.10 Impact of Resource Levelling

Table 2.13: Resource Levelling Impact Summary

Parameter	Before Levelling	After Levelling
Peak NLP Engineers Required	8	5
Peak Linguists Required	7	4
Resource Constraint Violations	3 periods	0 periods
Project Duration	218 days	232 days
Schedule Increase	-	14 days (6.4%)
Additional Cost	-	INR 3.5 Lakhs (if hiring)

Recommendation: Accept the 14-day schedule increase instead of hiring additional resources, as the project has no fixed external deadline. This saves INR 3.5 Lakhs.

2.4 Experiment 4: Project Crashing Techniques

2.4.1 Crash Analysis – Rationale

Project crashing is considered only if:

1. A fixed deadline of 200 days is imposed by the sponsor
2. The 14-day increase from resource levelling is unacceptable
3. Budget contingency (INR 4–5 Lakhs) can absorb crash costs

2.4.2 Activity Crash Options

Table 2.14: Activity Crash Options

Activity	**Normal (days)**	**Crash (days)**	**Crash Cost per Day (INR)**	**On Critical Path?**
C (Panini formalization)	53.00	40	25,000	Yes (critical)
D (Morphological analyzer)	45.83	35	30,000	Yes (critical)
E (Semantic parser)	40.83	32	28,000	Yes (critical)
G (API development)	20.83	16	22,000	Yes (critical)
H (Testing & validation)	28.67	22	18,000	Yes (critical)
B (Corpus annotation)	38.67	32	12,000	No (slack = 14 days)
F (Knowledge extraction)	28.67	24	15,000	No (slack = 12 days)

2.4.3 Least Cost Crashing – Step by Step

Initial Critical Path: A – C – D – E – G – H – I (Duration: 218 days)

Target: 200 days (crash requirement: 18 days)

Table 2.15: Least Cost Crashing Steps

Step	**Action**	**Days Crashed**	**Cumulative Crash**	**Cost (INR)**
1	Crash activity E (cheapest on critical path) – reduce from 41 to 36 days	5	5	$5 \times 28,000 = 1,40,000$

2	Crash activity D – reduce from 46 to 41 days	5	10	$5 \times 30,000 = 1,50,000$
3	Crash activity G – reduce from 21 to 18 days	3	13	$3 \times 22,000 = 66,000$
4	Crash activity H – reduce from 29 to 24 days	5	18	$5 \times 18,000 = 90,000$
Total Crash Cost				**4,46,000**

2.4.4 Crash Impact on Non-Critical Paths

After crashing critical activities by 18 days, the project duration becomes 200 days:

Table 2.16: Non-Critical Path Impact

Path	**Original Duration**	**Duration After Crashing**
A – B – D – E – G – H – I	201 days (via B)	183 days (B unchanged)
A – C – D – F – G – H – I	193 days	175 days

All non-critical paths remain below 200 days. No secondary critical path emerges.

2.4.5 Crash Summary

Table 2.17: Crash Summary

Parameter	**Normal Schedule**	**Crashed Schedule**
Project Duration	218 days	200 days
Total Crash Cost	-	INR 4,46,000
Schedule Reduction	-	18 days
Average Cost per Day Saved	-	INR 24,778
Budget Contingency Available	INR 4–5 Lakhs	INR 4–5 Lakhs
Crash Within Contingency?	-	Yes

2.4.6 Crash Recommendation

- **If deadline is 218 days (baseline)**: No crashing required – proceed with normal schedule

- **If deadline is 200 days (sponsor requirement)**: Execute crash plan (INR 4,46,000) – fully within contingency budget
- **If deadline is 210 days (intermediate)**: Crash only steps 1 and 2 (INR 2,90,000) for 10 days saving

2.4.7 Time-Cost Trade-off Curve Summary

Table 2.18: Time-Cost Trade-off Summary

Duration (days)	**Total Crash Cost (INR)**	**Marginal Cost (INR/day)**	**Feasibility**
218	0	-	Baseline
210	2,90,000	36,250	Recommended for moderate compression
205	3,56,000	13,200	Possible
200	4,46,000	18,000	Maximum recommended
195	5,80,000 (estimated)	26,800	Not recommended (diminishing returns)

2.4.8 Conclusion for Unit 2

The PERT analysis provides a 90.66% probability of completing the project within 230 days [2]. The CPM analysis identifies the critical path A–C–D–E–G–H–I with a baseline duration of 218 days. The project team structure includes 10 members with clearly defined roles and responsibilities. Resource levelling resolves linguist and engineer over-allocation by shifting non-critical activities, increasing duration by 14 days. Crashing can reduce the schedule to 200 days at an additional cost of INR 4,46,000, which is within the contingency budget. The recommended approach is to proceed with the baseline schedule unless a fixed deadline requires crashing [8].

Unit 3

Project Monitoring

3.1 Experiment 1: Project Performance Analysis

3.1.1 Cost, Time, Scope and Performance Monitoring

Project performance monitoring involves tracking actual progress against planned baselines across three key dimensions: cost, time, and scope [2]. For the Sanskrit NLP Framework project, Earned Value Management (EVM) is used as the primary performance monitoring technique.

3.1.2 Key Performance Indicators (KPIs)

The following KPIs have been established:

Table 3.1: Key Performance Indicators

KPI	**Target**	**Measurement Method**
Schedule Variance (SV)	$> -5\%$	EVMS – Compare EV vs PV
Cost Variance (CV)	$> -5\%$	EVMS – Compare EV vs AC
Schedule Performance Index (SPI)	> 0.95	EV / PV
Cost Performance Index (CPI)	> 0.95	EV / AC
Defect Density	< 2 per KLOC	Code review + testing
Requirement Stability Index (RSI)	$> 90\%$	(Initial requirements – Changes) / Initial
Linguist Validation Accuracy	$> 92\%$	Expert review of outputs
Customer Satisfaction (Post-UAT)	$> 85\%$	Survey of 10 beta users

3.1.3 Earned Value Analysis (EVA) – Month 6 Status

Reporting Period: End of Month 6 (after completion of Activity C – Panini rule formalization)

Table 3.2: Earned Value Analysis Parameters

Parameter	**Symbol**	**Value (INR Lakhs)**
Budget at Completion	BAC	37.00
Planned Value (cumulative)	PV	18.00
Earned Value (cumulative)	EV	16.50
Actual Cost (cumulative)	AC	19.00

3.1.4 Variance Analysis

$$\text{Cost Variance (CV)} = EV - AC = 16.50 - 19.00 = -2.50 \text{ Lakhs (Over budget)}$$

$$\text{Schedule Variance (SV)} = EV - PV = 16.50 - 18.00 = -1.50 \text{ Lakhs (Behind schedule)}$$

3.1.5 Cost and Schedule Performance Indices

$$\text{Cost Performance Index (CPI)} = \frac{EV}{AC} = \frac{16.50}{19.00} = 0.87$$

$$\text{Schedule Performance Index (SPI)} = \frac{EV}{PV} = \frac{16.50}{18.00} = 0.92$$

3.1.6 Forecasting

$$\text{Estimate at Completion (EAC)} = \frac{BAC}{CPI} = \frac{37.00}{0.87} = 42.53 \text{ Lakhs}$$

$$\text{Estimate to Complete (ETC)} = EAC - AC = 42.53 - 19.00 = 23.53 \text{ Lakhs}$$

$$\text{To-Complete Performance Index (TCPI)} = \frac{BAC - EV}{BAC - AC} = \frac{37.00 - 16.50}{37.00 - 19.00} = \frac{20.50}{18.00} = 1.14$$

3.1.7 Performance Improvement Plan

Based on the Month 6 analysis (CPI = 0.87, SPI = 0.92), the following corrective actions are recommended:

Table 3.3: Performance Improvement Plan

Action	Expected Impact	Cost
Add one junior NLP engineer to remaining activities	Increase SPI to 0.97	INR 60,000/month
Accelerate linguist review cycles (parallel reviews)	Reduce schedule slip by 5 days	INR 0 (process change)
Re-allocate contingency (INR 2 Lakhs) for critical path	Buffer against further delays	INR 2 Lakhs
Weekly EVA reporting for real-time tracking	Early detection of variances	INR 0

3.1.8 Project Controlling Techniques

Cost Controlling Techniques

- **Earned Value Management (EVM)**: Monthly tracking of CV and CPI
- **Budget Monitoring**: Weekly review of actual vs planned expenditure
- **Contingency Reserve Management**: Track reserve usage against risk triggers
- **Cost Baseline Control**: Formal change control for any budget adjustment

Schedule Controlling Techniques

- **Critical Path Monitoring**: Weekly tracking of critical path activities
- **Slack Management**: Monitor slack on non-critical paths
- **Milestone Tracking**: Dashboard of key milestone completion
- **Resource Loading**: Daily tracking of resource allocation

Scope Controlling Techniques

- **Requirements Traceability Matrix**: Link each requirement to deliverables
- **Change Control Board**: Formal review of all scope change requests

- **Scope Verification**: Customer acceptance at each milestone

3.1.9 Project Change Control

Table 3.4: Change Control Process

Step	Description
1. Change Request	Formal submission using Change Request Form
2. Impact Analysis	Assess impact on cost, schedule, scope, quality
3. CCB Review	Change Control Board meets weekly
4. Decision	Approve, reject, or defer with conditions
5. Implementation	Update project documents and communicate
6. Verification	Verify change was implemented correctly

Change Control Board (CCB) Members:

- Project Manager (Chair)
- Lead Sanskrit Linguist
- Lead NLP Engineer
- Quality Assurance Lead
- Sponsor Representative (for major changes)

3.1.10 Project Quality Controlling Techniques

Table 3.5: Quality Control Techniques

Technique	Application	Frequency
Code Reviews	Peer review of all code changes	Daily
Unit Testing	Automated pytest suite	Per commit
Integration Testing	API endpoint validation	Per sprint
Linguist Validation	Expert review of Panini rules	Weekly
Defect Tracking	JIRA for bug tracking	Continuous
Statistical Sampling	Random validation of outputs	Monthly

3.1.11 Revised Forecast After Corrective Actions

Table 3.6: Revised Forecast

Parameter	Before Correction	After Correction
EAC (INR Lakhs)	42.53	39.80
Projected Completion Delay	18 days	8 days
CPI (forward-looking)	0.87	0.94
SPI (forward-looking)	0.92	0.97

3.2 Experiment 2: Risk Management and Mitigation

3.2.1 Project Risk Analysis

Risk analysis involves identifying, assessing, and prioritizing risks that could affect project objectives [2]. Six key risks were identified through brainstorming sessions:

Table 3.7: Risk Identification and Assessment

Risk ID	Risk Description	Probability (P)	Impact (I)	Risk Score (P × I)
R01	Shortage of Sanskrit linguists with NLP knowledge	High (0.7)	High (0.8)	0.56
R02	Panini rule formalization takes longer than expected	Medium (0.5)	High (0.8)	0.40
R03	Low accuracy (<85%) on complex sandhi phenomena	Medium (0.5)	High (0.7)	0.35
R04	Cloud cost overrun due to large corpus processing	High (0.6)	Medium (0.5)	0.30
R05	Low adoption by academic institutions	Low (0.3)	High (0.7)	0.21
R06	Dependency on third-party Sanskrit corpus licenses	Medium (0.4)	Medium (0.5)	0.20

3.2.2 Risk Matrix (Probability-Impact Grid)

	Low Impact	Medium Impact	High Impact
High Probability	–	R04	R01
Medium Probability	–	R06	R02, R03
Low Probability	–	–	R05

3.2.3 Risk Response Strategies

Table 3.8: Risk Response Strategies

Risk ID	Response Strategy	Specific Actions	Owner
R01	Mitigate	Partner with 2 Sanskrit universities for part-time linguists; Budget INR 3 Lakhs for honorarium	Project Manager
R02	Mitigate	Develop incremental rule engine; Release v1 with 500 sutras, v2 with 1000 sutras	NLP Lead
R03	Mitigate	Build gold-standard test dataset of 5000 words with expert validation before coding begins	QA Lead
R04	Mitigate	Use spot instances for non-critical batch processing; Set up budget alerts at INR 3 Lakhs	Cloud Engineer
R05	Accept + Contingency	Offer free 2-year pilot to top 5 institutions; Build case studies	Product Manager
R06	Transfer	Use open-source Sanskrit corpora (DCS, GRETIL) as fallback; Negotiate perpetual license	Legal + Data Lead

3.2.4 Risk Contingency Reserve

Table 3.9: Risk Contingency Reserve

Risk Category	Contingency Allocated (INR Lakhs)	Notes

Linguist shortage (R01)	3.00	Honorarium for 3 part-time linguists
Schedule overrun (R02, R03)	2.00	Overtime and additional resources
Infrastructure (R04)	1.00	Buffer for cloud cost overrun
Legal / Licensing (R06)	0.50	Fallback corpus licenses
Unforeseen risks	1.00	General contingency
Total Contingency Reserve	**7.50**	Within 15-20% of total budget

3.3 Experiment 3: RMMM Plan, Configuration Management & GitHub

3.3.1 RMMM (Risk Mitigation, Monitoring & Management) Plan

Monitoring Frequency

Table 3.10: RMMM Monitoring Schedule

Activity	Frequency	Responsible
Risk review meeting	Bi-weekly (alternate Wednesdays)	Project Manager
Risk register update	Weekly (every Friday)	Risk Owner
Mitigation action tracking	Weekly	Project Manager
Trigger monitoring (linguist availability)	Daily	Team Lead
Early warning indicator review	Monthly	Steering Committee

Risk Triggers and Thresholds

Table 3.11: Risk Triggers and Thresholds

Risk ID	Trigger	Response Activation Threshold
R01	Less than 2 linguists available for 2 consecutive weeks	Immediately escalate to sponsor

R02	Panini rule implementation slips by >10 days	Crash activity C (INR 25,000/day)
R03	Sandhi accuracy <80% on intermediate test	Add second linguist to validation
R04	Monthly cloud cost exceeds INR 1 Lakh	Migrate non-critical jobs to spot instances
R05	Zero paid licenses after 6 months of launch	Launch promotional pricing (50% discount)

3.3.2 Configuration Management Plan

Configuration Items (CIs)

The following items are under configuration management:

1. Panini rule base (XML files) – versioned as ruleset_v1.0, v1.1, etc.
2. Morphological analyzer source code (Python) – GitHub repository
3. Semantic parser models (PyTorch weights) – versioned with model_v1.0.pt
4. API specifications (OpenAPI YAML) – versioned with API version
5. Test datasets and gold standards – versioned with dataset_v1.0
6. Documentation (User guide, API reference) – versioned with release
7. Docker images and deployment scripts – versioned with build tag

3.3.3 Software Configuration Management: GitHub

Repository Structure

```
/sanskrit-nlp-framework/
+-- .github/
    +-- workflows/
        +-- ci.yml
        +-- deploy.yml
+-- src/
    +-- morphological_analyzer/
    +-- semantic_parser/
    +-- knowledge_extraction/
    +-- api/
+-- tests/
```

```
    +-- unit/
    +-- integration/
    +-- fixtures/
+-- data/
    +-- corpora/
    +-- rules/
    +-- gold_standards/
+-- docs/
    +-- api/
    +-- user_guide/
+-- models/
    +-- trained/
    +-- checkpoints/
+-- docker/
    +-- Dockerfile
    +-- docker-compose.yml
+-- requirements.txt
+-- README.md
+-- CHANGELOG.md
```

Branching Strategy (Git Flow)

Table 3.12: Git Flow Branching Strategy

Branch	Purpose	Protection Rules
main	Production-ready code	PR required; 2 approvals; all CI passes
develop	Integration branch	PR required; 1 approval; CI passes
feature/*	New features	No protection; deleted after merge
release/*	Release preparation	PR to main and develop
hotfix/*	Urgent production fixes	PR to main; 1 approval

CI/CD Pipeline (GitHub Actions)

Table 3.13: CI/CD Pipeline Stages

Stage	Actions	Trigger
Lint	flake8, black, isort	On push to any branch

Unit Test	pytest (minimum 80% coverage)	On push to feature/develop
Integration Test	API endpoint tests	On PR to develop
Build	Docker image build	On merge to develop
Deploy (Staging)	Deploy to staging environment	On merge to develop
Deploy (Production)	Deploy to production	On tag release/v*

3.3.4 Versioning Convention

Semantic Versioning (SemVer 2.0.0): **Major.Minor.Patch**

- **Major**: Incompatible API changes, complete Panini rule overhaul
- **Minor**: Backward-compatible functionality additions (new sutras, new API endpoints)
- **Patch**: Backward-compatible bug fixes, documentation updates

Example: `v1.0.0` – First production release with 500 Panini sutras

3.4 Experiment 4: Unit Testing with Test Cases

3.4.1 Testing Strategy Overview

Table 3.14: Testing Strategy

Test Level	Framework	Target Coverage
Unit Testing	pytest (Python)	85%
Integration Testing	pytest + FastAPI TestClient	90% of API endpoints
Regression Testing	pytest + GitHub Actions	100% of prior tests
Accuracy Validation	Custom validation suite + Linguist review	92% precision
Performance Testing	locust	< 2 sec/verse

3.4.2 Sample Test Case 1: Basic Noun (Nominative Singular)

Table 3.15: Test Case TC-MORPH-001

Test Case ID	TC-MORPH-001
Test Description	Validate morphological analysis of a simple Sanskrit noun in nominative singular
Input	raama.h (raama with visarga)
Expected Output	{word: ”raama.h”, stem: ”raama”, case: ”nominative”, number: ”singular”, gender: ”masculine”}
Actual Output	{word: ”raama.h”, stem: ”raam”, case: ”nominative”, number: ”singular”, gender: ”masculine”}
Status	FAIL
Root Cause	Sandhi rule for visarga removal incorrectly applied
Fix	Update sandhi rule: remove visarga before applying suffix stripping

3.4.3 Sample Test Case 2: Present Tense Verb

Table 3.16: Test Case TC-MORPH-015

Test Case ID	TC-MORPH-015
Test Description	Validate morphological analysis of present tense verb (3rd person singular)
Input	gacchati
Expected Output	{root: ”gam”, tense: ”present”, person: ”3rd”, number: ”singular”, voice: ”active”, pada: ”parasmaipada”}
Actual Output	{root: ”gacch”, tense: ”present”, person: ”3rd”, number: ”singular”, voice: ”active”, pada: ”parasmaipada”}
Status	FAIL
Root Cause	Irregular root transformation ”gam → gaccha” not in rule base
Fix	Add irregular verb lookup table for root ”gam” and 10 other common roots

3.4.4 Sample Test Case 3: Compound Word (Sandhi) – PASS

Table 3.17: Test Case TC-MORPH-042 (Passing)

Test Case ID	TC-MORPH-042
Test Description	Validate sandhi splitting and analysis of compound word
Input	devaalaya.h

Expected Output	{components: ["deva", "aalaya"], split_rule: "guna sandhi", analysis: ["deva: masc nom sg", "aalaya: masc nom sg"]}
Actual Output	{components: ["deva", "aalaya"], split_rule: "guna sandhi", analysis: ["deva: masc nom sg", "aalaya: masc nom sg"]}
Status	PASS

3.4.5 Test Execution Summary (Sprint 4)

Table 3.18: Test Execution Summary

Module	**Test Cases**	**Passed**	**Failed**
Morphological Analyzer	45	38	7
Semantic Parser	25	21	4
Knowledge Extraction	15	13	2
API Endpoints	12	12	0
Integration	8	8	0
Total	**105**	**92**	**13**

Overall Pass Rate: $\frac{92}{105} \times 100 = 87.6\%$

3.4.6 Defect Analysis

Table 3.19: Defect Analysis by Severity

Severity	**Description**	**Count**	**Action**
Critical	Prevents core functionality (morphological analysis)	2	Fix immediately; re-run tests
Major	Affects accuracy for specific cases	5	Fix within sprint
Minor	Edge cases or documentation issues	6	Fix in next sprint

3.4.7 Corrective Actions Based on Unit Testing

1. **Critical Defect 1** (Stemming error for visarga-ending words): Update sandhi preprocessing module – estimated 4 hours

2. **Critical Defect 2** (Root extraction for irregular verbs): Implement irregular verb table with 50 common roots – estimated 8 hours

3. **Major Defects**: Add 15 additional sandhi rules for common patterns – estimated 2 days

4. **Regression Testing**: After fixes, re-run all 105 test cases before next deployment

3.4.8 Quality Metrics Dashboard

Table 3.20: Quality Metrics Dashboard

Metric	Target	Current	Status
Unit Test Pass Rate	95%	87.6%	Below Target
Code Coverage	85%	79%	Below Target
Defect Density	< 2/KLOC	1.8/KLOC	On Track
Mean Time to Fix (Critical)	< 24 hrs	18 hrs	On Track
Linguist Validation Accuracy	92%	89%	Below Target

3.4.9 Conclusion for Unit 3

The project is currently slightly behind schedule (SPI = 0.92) and over budget (CPI = 0.87) at Month 6. Earned Value Analysis indicates an EAC of INR 42.53 Lakhs, requiring corrective actions [2]. Corrective actions including additional NLP engineer and re-allocated contingency are expected to improve performance to SPI = 0.97 and CPI = 0.94. Risk management has identified six key risks with mitigation strategies and a contingency reserve of INR 7.5 Lakhs. Configuration management is fully implemented using GitHub with Git Flow branching and CI/CD pipelines. Unit testing shows 87.6% pass rate with two critical defects being addressed immediately. Quality metrics indicate need for improved test coverage and linguist validation accuracy before production release [3].

Unit 4

Project Closing Phase & Agile Frameworks

4.1 Experiment 1: Project Audits

4.1.1 Project Audits: Objectives and Goals

A project audit is a systematic review of a project's management, processes, and deliverables to assess compliance with standards, identify improvement opportunities, and verify project status [2]. The primary objectives of audits for the Sanskrit NLP Framework project are:

1. Verify compliance with Panini grammar accuracy standards
2. Assess adherence to schedule and budget baselines
3. Identify process improvements for future sprints
4. Validate quality metrics and test coverage
5. Ensure configuration management discipline

4.1.2 Types of Audits

Table 4.1: Types of Audits for Sanskrit NLP Project

Audit Type	**Frequency / Trigger**	**Responsible Party**
Technical audit (code quality, Panini rule accuracy)	End of each sprint	QA Lead + Linguist
Financial audit	Quarterly	Finance Controller

Security audit (API, user data)	Before v1.0 release	External Security Consultant
Process audit (Agile compliance)	Monthly	Scrum Master
Quality assurance audit (test coverage, documentation)	Bi-weekly	QA Lead
Configuration management audit	Per release	Configuration Manager

4.1.3 Audit Checklist (Sample – Technical Audit)

The following checklist is used for technical audits at the end of each sprint:

1. Are Panini sutras correctly mapped to code? (Validation by linguist)
2. Is test coverage $> 85\%$? (Measured by pytest-cov)
3. Are all configuration items version controlled in GitHub?
4. Is code duplication $< 5\%$? (Measured by radon or flake8)
5. Are API responses under 2 seconds for 100-word input?
6. Is there at least one linguist review per sprint?
7. Are all critical defects fixed before release?
8. Is documentation updated for all new features?

4.1.4 Audit Findings (Sprint 3)

Table 4.2: Audit Findings and Corrective Actions

Area	**Issue**	**Severity**	**Corrective Action**
Panini rule mapping	8 sutras incorrectly implemented	High	Re-implement with linguist pair review
Test coverage	Only 72% (target 85%)	Medium	Add 20 more unit tests for edge cases

Documentation	API docs missing authentication section	Low	Complete within 3 days
Code duplication	7% duplication in morphological analyzer	Medium	Refactor shared functions
CI/CD pipeline	Deployment step failing intermittently	High	Fix environment variables

4.1.5 Audit Remediation Tracking

Table 4.3: Audit Remediation Tracking

Issue	Assigned To	Due Date	Status	Verified By
8 sutras incorrect	NLP Engineer 1 + Linguist	Sprint 3 + 2 days	Closed	QA Lead
Test coverage low	NLP Engineer 2	Sprint 4	In Progress	-
API documentation	NLP Engineer 3	Sprint 3 + 3 days	Closed	PM
Code duplication	NLP Engineer 1	Sprint 4	In Progress	-
CI/CD pipeline	DevOps Engineer	Sprint 3 + 1 day	Closed	QA Lead

4.1.6 Project Termination

Project termination occurs after successful completion of all deliverables and acceptance criteria. The termination process for the Sanskrit NLP Framework includes:

1. **Product Verification**: Final accuracy validation by Sanskrit linguists
2. **Documentation Handover**: Complete technical and user documentation
3. **Knowledge Transfer**: Training sessions for client technical team
4. **Financial Closure**: Final billing and budget reconciliation
5. **Lessons Learned**: Documentation of successes and improvement areas
6. **Stakeholder Sign-off**: Formal acceptance from sponsor and users

4.1.7 Software Testing and Testing Strategies

Types of Testing

Table 4.4: Types of Testing for Sanskrit NLP Project

Test Type	**Purpose**	**Tool/Method**
Unit Testing	Validate individual components	pytest
Integration Testing	Test component interactions	pytest + FastAPI
System Testing	End-to-end functionality	Custom test suite
Acceptance Testing	Validate against requirements	Linguist validation
Regression Testing	Ensure no new defects	GitHub Actions
Performance Testing	Verify response times	locust
Security Testing	Identify vulnerabilities	bandit, OWASP ZAP
Usability Testing	Evaluate user experience	User feedback sessions

4.1.8 Evaluation of Project

Project evaluation is conducted at three levels:

Level 1: Technical Evaluation

- Morphological analysis accuracy $\geq 92\%$
- API response time ≤ 2 seconds
- Successful batch processing of Mahabharata corpus

Level 2: Managerial Evaluation

- Schedule variance within $\pm 5\%$
- Cost variance within $\pm 5\%$
- Stakeholder satisfaction $\geq 85\%$

Level 3: Business Evaluation

- Adoption by ≥ 15 academic institutions in Year 2
- Revenue generation of INR 22.5 Lakhs in Year 2
- Positive return on investment by Year 3

4.2 Experiment 2: Agile Approaches Framework – Sprint Planning, Review

4.2.1 Introduction to Agile Process Development

Agile software development is an iterative approach that emphasizes flexibility, collaboration, and customer feedback [14]. Unlike traditional waterfall models, Agile delivers working software in small, frequent increments.

4.2.2 Manifesto of Agile Process

The Agile Manifesto [14] values:

1. **Individuals and interactions** over processes and tools
2. **Working software** over comprehensive documentation
3. **Customer collaboration** over contract negotiation
4. **Responding to change** over following a plan

4.2.3 Agile Principles

The 12 Agile principles applied to the Sanskrit NLP project:

Table 4.5: Agile Principles and Their Application

Principle	Application in Sanskrit NLP Project
1	Highest priority: satisfy customer through early and continuous delivery of Panini rule engine
2	Welcome changing requirements even late in development (new sutras can be added)
3	Deliver working software frequently (weekly API releases)
4	Business people and linguists work together daily
5	Build projects around motivated individuals – trust the team
6	Most efficient communication is face-to-face (daily stand-ups)
7	Working software is the primary measure of progress
8	Agile processes promote sustainable development – no overtime culture

9	Continuous attention to technical excellence and good design
10	Simplicity – maximize work not done
11	Best architectures emerge from self-organizing teams
12	Team reflects on how to become more effective (sprint retrospectives)

4.2.4 Agile Practices and Methodologies

For the Sanskrit NLP project, the following Agile practices are adopted:

- **Scrum**: 2-week sprints with defined roles and ceremonies
- **Kanban**: Visual workflow management using story boards
- **Extreme Programming (XP)**: Pair programming, TDD, continuous integration
- **Lean**: Eliminate waste, amplify learning, decide as late as possible

4.2.5 Agile Framework – Phases of Development

Table 4.6: Agile Development Phases

Phase	Activities	Output
Envision	Product vision, roadmap, initial backlog	Product vision statement
Speculate	Release planning, sprint planning	Sprint backlog
Explore	Development, testing, daily stand-ups	Working increment
Adapt	Sprint review, retrospective	Improvements
Close	Release, deployment, handover	Shippable product

4.2.6 Relationship Between Conventional and Agile

Table 4.7: Conventional vs Agile Project Management

Aspect	Conventional (Waterfall)	Agile (Scrum/XP)

Requirements	Fixed at start	Evolving throughout
Delivery	Single final delivery	Frequent incremental delivery
Customer involvement	At milestones only	Continuous throughout
Documentation	Comprehensive	Just-in-time
Team structure	Hierarchical	Self-organizing
Change management	Formal CCB	Welcomed, prioritized in backlog
Testing	End of project	Continuous throughout
Risk management	Upfront planning	Continuous adaptation

4.2.7 IT Service Management Lifecycle

The project aligns with ITIL (Information Technology Infrastructure Library) service management lifecycle:

1. **Service Strategy**: Define value proposition for academic users
2. **Service Design**: Design API, documentation, support model
3. **Service Transition**: Deploy using CI/CD pipeline
4. **Service Operation**: Monitor API performance, user support
5. **Continual Improvement**: Sprint retrospectives, user feedback

4.2.8 Sprint Structure for Sanskrit NLP Project

Table 4.8: Sprint Structure

Parameter	**Value**
Sprint duration	2 weeks (10 working days)
Team size	6 members (1 PM, 3 NLP engineers, 1 linguist, 1 QA)
Velocity (planned)	30 story points per sprint
Velocity (actual average)	26 story points per sprint
Product Owner	Lead Sanskrit Linguist (Dr. Sharma)
Scrum Master	Project Manager
Definition of Ready	Story has acceptance criteria, linguist approval, estimated

Definition of Done	Code reviewed, tests pass, documentation updated, deployed to staging

4.2.9 Sprint Planning (Sprint 4 Example)

Table 4.9: Sprint Planning Example

User Story	Story Points	Assigned To
As a researcher, I can submit a Sanskrit verse and get morphological analysis	8	NLP Engineer 1
As a linguist, I can add custom sandhi rules via configuration file	5	NLP Engineer 2
As a developer, I can run unit tests locally before commit	3	NLP Engineer 3
As a student, I can view parse tree in visual format	8	NLP Engineer 1 + 2
As a QA engineer, I can run automated regression suite	4	QA Lead
As a product owner, I can review rule coverage metrics	2	NLP Engineer 3
Total Planned	**30**	

4.2.10 Sprint Review (Sprint 4 – Actual)

Table 4.10: Sprint Review Results

User Story	Points	Status
Morphological analysis API endpoint	8	Completed
Custom sandhi rules configuration	5	Completed (except documentation)
Local unit test script	3	Completed
Visual parse tree	8	Partially completed (moved to Sprint 5)
Automated regression suite	4	Completed

Rule coverage metrics dashboard	2	Completed
Total Completed	**22**	

Velocity achieved: 22 points (vs planned 30 points)

4.3 Experiment 3: Daily Scrum Planning, Story Boards Creation, Tracking Progress

4.3.1 Daily Scrum (Stand-up) Format

- **Time**: 9:30 AM daily (maximum 15 minutes)
- **Platform**: Microsoft Teams / Zoom
- **Participants**: Entire Scrum team (6 members)
- **Scrum Master facilitates**

Each team member answers three questions:

1. What did I complete yesterday?
2. What will I do today?
3. Are there any blockers?

4.3.2 Sample Daily Scrum (Sprint 4, Day 3)

Table 4.11: Daily Scrum Example

Member	**Update**	**Blockers**
NLP Engineer 1	Completed sandhi rule parser; today: integrate with API	Awaiting linguist review (expected today)
NLP Engineer 2	Fixed 4 failing unit tests; today: add 5 more tests for edge cases	None
NLP Engineer 3	Completed API documentation; today: start metrics dashboard	None

Linguist	Reviewed 10 Panini sutras; today: review sandhi rules for Engineer 1	None
QA Lead	Tested 2 API endpoints; today: automate regression suite	None
Project Manager	Updated risk register; today: stakeholder status report	None

4.3.3 Story Board Creation (Kanban Style)

Table 4.12: Story Board (Kanban)

TO DO	**IN PROGRESS**	**REVIEW**	**DONE**
Visual parse tree (8 pts)	Sandhi rule API (80% complete)	Custom rules config (In review)	Morph analyzer endpoint (8 pts)
Documentation review	Unit test auto (60% complete)	-	Local test script (3 pts)
-	-	-	Regression suite (4 pts)

4.3.4 Tracking Progress – Sprint Burndown Chart

Table 4.13: Sprint Burndown Data (Sprint 4)

Day	**Remaining Points**	**Ideal Remaining**	**Variance**
0 (Planning)	30	30	0
1	28	27	+1
2	25	24	+1
3	22	21	+1
4	20	18	+2
5	18	15	+3
6	14	12	+2
7	10	9	+1

8	8	6	+2
9	4	3	+1
10 (Review)	2	0	+2

Analysis: Team consistently above ideal burndown line due to incomplete visual parse tree story. Recommended action: Split large stories in future sprints.

4.3.5 Cumulative Flow Diagram (CFD) Metrics

Table 4.14: Cumulative Flow Diagram Data

Day	To Do	In Progress	Review	Done
0	30	0	0	0
3	18	8	2	2
6	10	6	4	10
10	2	0	2	26

4.3.6 Sprint Retrospective – Key Action Items (Sprint 4)

1. Add linguist earlier in code review cycle (before implementation, not after)
2. Reduce maximum story size from 8 points to 5 points for better predictability
3. Automate sandhi rule validation using unit tests
4. Schedule daily 15-minute linguist-engineer sync for complex rules
5. Split visual parse tree story into 2 smaller stories (backend + frontend)

4.4 Experiment 4: Agile Approaches – Extreme Programming, Small Releases, Scrum, Lean

4.4.1 Extreme Programming (XP) Practices Adopted

Table 4.15: XP Practices for Sanskrit NLP Project

XP Practice	How Implemented	Benefit Observed

Pair programming	Complex Panini rule implementation with 2 engineers + 1 linguist	40% reduction in rule errors
Test-driven development (TDD)	Write test first for each morphological rule before implementation	Higher confidence in accuracy
Small releases	Weekly release of API improvements to staging	Faster feedback from linguists
Continuous integration	GitHub Actions runs all tests on every commit	Early defect detection (avg 2 hours)
Refactoring	Monthly code cleanup session	Code duplication reduced from 7% to 4%
Collective code ownership	Any engineer can fix any bug; no silos	Faster bug resolution (avg 6 hours)
On-site customer	Sanskrit linguist available full-time during sprints	Immediate clarification of rules
Sustainable pace	40-hour weeks; no overtime except critical fixes	Consistent velocity (22-28 points)

4.4.2 Small Release Strategy

Table 4.16: Small Release Plan

Release	**Content**	**Frequency**
v0.1.0 (Alpha)	Basic morphological analyzer (100 sutras)	Week 4
v0.2.0 (Alpha)	Sandhi splitting for common patterns	Week 6
v0.3.0 (Beta)	Semantic parser for simple verses	Week 8
v0.4.0 (Beta)	Knowledge extraction module	Week 10
v0.5.0 (Beta)	REST API with 3 endpoints	Week 12
v1.0.0 (Production)	Complete framework with documentation	Week 18

4.4.3 Scrum Framework Integration

Table 4.17: Scrum + XP Integration

Scrum Element	XP Practice Integrated	Outcome
Sprint planning	User stories with acceptance tests (TDD)	Clear definition of done
Daily scrum	Pair programming coordination	Better task allocation
Sprint review	Small release demo to stakeholders	Immediate feedback
Sprint retrospective	Refactoring and process improvement planning	Continuous improvement
Product backlog	Continuous prioritization by linguist (on-site customer)	Highest value delivered first

4.4.4 Lean Principles Applied

Table 4.18: Lean Principles in Sanskrit NLP Project

Lean Principle	Application	Measured Impact
Eliminate waste	Auto-generate documentation from code; no duplicate rule specification	Saved 15 person-hours per sprint
Amplify learning	Weekly knowledge transfer sessions between linguists and engineers	Reduced rule misinterpretation by 60%
Decide as late as possible	API design finalized after 3 sprints of experimentation	Avoided 2 major redesigns
Deliver as fast as possible	CI/CD pipeline deploys to staging within 1 hour of commit	Lead time reduced from 5 days to 4 hours
Empower the team	Linguists can directly fix rule mapping via config files	No dependency on engineers for rule changes
Build integrity in	Automated sandhi validation before merge	Defect escape rate reduced to 12%
See the whole	Burndown chart + CFD visible to all stakeholders	Improved transparency and trust

4.4.5 Scrum, XP, and Lean Comparison

Table 4.19: Comparison of Agile Methodologies

Methodology	Primary Focus	Key Practices Used
Scrum	Project management	Sprints, roles, ceremonies, backlog
XP	Engineering practices	TDD, pair programming, CI, refactoring
Lean	Waste elimination	Value stream mapping, pull systems, Kaizen

4.4.6 Benefits of Hybrid Agile Approach

- **Improved Quality**: TDD and pair programming reduced defect density by 40%
- **Faster Delivery**: Small releases enabled weekly feedback cycles
- **Higher Customer Satisfaction**: Linguist involvement increased accuracy validation
- **Better Adaptability**: Changes welcomed even late in development
- **Reduced Waste**: Lean principles eliminated redundant documentation

4.4.7 Conclusion for Unit 4

Project audits conducted at sprint boundaries identified and resolved critical issues including 8 incorrect Panini sutras and CI/CD pipeline failures. The Agile transformation using Scrum with 2-week sprints yielded measurable improvements: velocity increased from 18 to 26 story points over 8 weeks [14]. Daily Scrum meetings ensured effective coordination, while story boards and burndown charts provided visual progress tracking. XP practices, particularly pair programming for complex Panini rules, reduced implementation errors from 15% to 6%. Lean principles eliminated waste and accelerated delivery. The hybrid Agile approach successfully balanced the need for rigorous Panini grammar accuracy with flexible, iterative development [15].

Unit 5

Scrum Methodology, DevOps & Extreme Programming

5.1 Experiment 1: Scrum Method and Framework

5.1.1 Scrum Framework and Its Scope

Scrum is an Agile framework for developing, delivering, and sustaining complex products [15]. It is lightweight, simple to understand, but difficult to master. For the Sanskrit NLP Framework project, Scrum provides the structural backbone for iterative development while maintaining rigorous quality standards for Panini grammar accuracy.

The scope of Scrum in this project includes:

- Managing development of morphological analyzer and semantic parser
- Coordinating between NLP engineers and Sanskrit linguists
- Ensuring regular delivery of working increments
- Providing transparency to stakeholders

5.1.2 Project Management Activities in Scrum

Table 5.1: Scrum Project Management Activities

Activity	**Frequency**	**Purpose**
Sprint Planning	Start of each sprint	Define sprint goal and select backlog items
Daily Scrum	Every day (15 min)	Synchronize activities and identify blockers

Sprint Review	End of each sprint	Inspect increment and adapt backlog
Sprint Retrospective	End of each sprint	Improve processes and team performance
Backlog Refinement	Throughout sprint	Clarify and estimate future backlog items

5.1.3 Sprint Backlog

The Sprint Backlog is a highly visible, real-time picture of the work the Development Team plans to accomplish during the Sprint [15].

Table 5.2: Sprint Backlog Example (Sprint 5)

Task ID	**Description**	**Owner**	**Estimate (hrs)**
SB-01	Implement remaining 200 Panini sutras	NLP Eng 1 + Linguist	40
SB-02	Fix sandhi splitting for 15 edge cases	NLP Eng 2	16
SB-03	Complete visual parse tree backend	NLP Eng 1	8
SB-04	Add authentication to API	NLP Eng 3	8
SB-05	Write integration tests for new endpoints	QA Lead	8
SB-06	Update API documentation	NLP Eng 3	4
Total			**84 hours**

5.1.4 Sprint Review

The Sprint Review is held at the end of the Sprint to inspect the Increment and adapt the Product Backlog [15].

Table 5.3: Sprint Review Structure

Component	**Duration**	**Participants**
Product Owner presents completed work	15 min	All stakeholders

Team demonstrates new functionality	20 min	Development Team
Stakeholders provide feedback	15 min	Stakeholders
Product Owner updates backlog	10 min	Product Owner + Team

5.1.5 Retrospective Perspective

The Sprint Retrospective is an opportunity for the Scrum Team to inspect itself and create a plan for improvements to be enacted during the next Sprint [15].

Table 5.4: Sprint Retrospective Structure

Phase	**Activity**	**Output**
Set the stage	Review sprint goals and outcomes	Shared context
Gather data	What went well? What went wrong?	Data points
Generate insights	Root cause analysis	Improvement hypotheses
Decide what to do	Select top 1-3 improvements	Action items
Close the retrospective	Review action items	Commitment to next sprint

5.1.6 Best Practices of Scrum

Table 5.5: Scrum Best Practices for Sanskrit NLP Project

Best Practice	**Implementation**
Definition of Done	Code reviewed, tests pass, documentation updated, linguist validated
Time-boxed events	Sprint: 2 weeks, Daily Scrum: 15 min, Planning: 2 hours
Cross-functional team	Engineers + Linguist + QA working together
Visible progress	Story board and burndown chart displayed publicly
Sustainable pace	No overtime; 40-hour work weeks
Impediment removal	Scrum Master responsible for blocker resolution
Continuous improvement	Action items from retrospectives tracked and implemented

5.1.7 Roles in Scrum

Table 5.6: Scrum Roles and Responsibilities

Role	Person	Key Responsibilities
Product Owner	Lead Sanskrit Linguist (Dr. Sharma)	Maximize value; manage backlog; prioritize Panini sutras
Scrum Master	Project Manager	Facilitate ceremonies; remove blockers; coach team
Development Team	3 NLP Engineers + 1 QA + 1 Linguist	Deliver potentially shippable increment each sprint

5.1.8 Slack in Scrum

Slack refers to buffer time intentionally built into the sprint to handle unforeseen issues, technical debt, or improvement work.

Table 5.7: Slack Allocation in Sprints

Sprint Purpose	Total Capacity (hrs)	Slack (hrs)
Sprint 1 Team onboarding	80	8
Sprint 2 Learning curve	80	4
Sprint 3 Technical debt	80	4
Sprint 4 Documentation	80	4
Sprint 5 Integration testing	80	8

5.1.9 Ten Minute Build

Ten-minute build is an XP practice that ensures the entire system can be built and all tests can run within 10 minutes [14].

Table 5.8: Ten Minute Build Implementation

Component Target	Current Time
Code compilation < 1 minute	45 seconds
Unit tests (105 tests) < 2 minutes	90 seconds
Integration tests < 2 minutes	120 seconds
Lint and style checks < 30 seconds	30 seconds
Docker image build < 4 minutes	180 seconds
Total **Under 10 min**	**465 seconds (7.75 min)**

5.1.10 Continuous Integration

Continuous Integration (CI) is the practice of merging all developer working copies to a shared mainline several times a day [14].

Table 5.9: CI Pipeline Implementation

Stage	Tool	Trigger
Code commit	Git	Developer push
Linting	flake8, black	On push to any branch
Unit testing	pytest	On push to feature/develop
Integration testing	pytest + API	On PR to develop
Build	Docker	On merge to develop
Deploy (staging)	GitHub Actions	On merge to develop

5.2 Experiment 2: Weekly Cycle, Pair Programming, Coding Standards

5.2.1 Weekly Cycle Structure

Table 5.10: Weekly Agile Cycle

Day	Morning (9:30-12:30)	Afternoon (1:30-5:30)
Monday	Sprint Planning (first week) / Daily Scrum + Coding	Coding + Pair programming sessions
Tuesday	Daily Scrum + Coding	Code review + Documentation
Wednesday	Daily Scrum + Pair programming (complex rules)	Unit testing + Integration testing
Thursday	Daily Scrum + Coding	Code review + Linguist validation
Friday	Daily Scrum + Backlog refinement	Sprint Review (last week) / Retrospective

5.2.2 Pair Programming Schedule

Table 5.11: Pair Programming Schedule

Session	Pair	Focus Area
Monday 2-4 PM	NLP Engineer 1 + NLP Engineer 2	Sandhi rule implementation
Tuesday 2-4 PM	NLP Engineer 2 + Linguist	Panini sutra validation
Wednesday 10-12 PM	NLP Engineer 3 + NLP Engineer 1	API error handling
Thursday 2-4 PM	NLP Engineer 1 + Linguist	Complex morphological cases
Friday 10-12 PM	NLP Engineer 2 + NLP Engineer 3	Performance optimization

5.2.3 Coding Standards (Python – PEP 8 with Extensions)

Table 5.12: Coding Standards

Standard	Description	Tool
Naming conventions	snake_case for variables/functions; PascalCase for classes	flake8
Line length	Maximum 88 characters	black (line length 88)
Docstrings	Google style docstrings for all public functions	pydocstyle

Type hints	Required for all function parameters and returns	mypy (strict mode)
Imports	Grouped: standard library, third-party, local	isort
Testing	Every function must have corresponding unit test	pytest (minimum 80% coverage)

5.2.4 Code Quality Enforcement

```
Pre-commit hooks configuration (.pre-commit-config.yaml):
  - flake8: style guide enforcement (zero warnings)
  - black: automatic code formatting
  - isort: import ordering
  - mypy: static type checking (strict mode)
  - pytest: unit tests (85% coverage minimum)
```

5.3 Experiment 3: DevOps using Docker

5.3.1 Introduction to DevOps

DevOps is a set of practices that combines software development (Dev) and IT operations (Ops) to shorten the development lifecycle while delivering features, fixes, and updates frequently [14]. For the Sanskrit NLP project, DevOps enables:

- Consistent environment across development, staging, and production
- Automated testing and deployment
- Faster feedback loops
- Reduced deployment failures

5.3.2 Docker Implementation Strategy

The Sanskrit NLP Framework is containerized using Docker to ensure consistent execution across environments.

Dockerfile (Production)

```
FROM python:3.10-slim AS base
WORKDIR /app
COPY requirements.txt .
```

```
RUN pip install --no-cache-dir -r requirements.txt
COPY src/ ./src/
COPY models/ ./models/
COPY data/rules/ ./data/rules/
EXPOSE 8000
CMD ["uvicorn", "src.api.main:app", "--host", "0.0.0.0", "--port"
, "8000"]
```

Docker Compose (Development Environment)

```
version: '3.8'
services:
  api:
    build: .
    ports:
      - "8000:8000"
    volumes:
      - ./src:/app/src
      - ./data:/app/data
    environment:
      - ENVIRONMENT=development
      - LOG_LEVEL=debug
    depends_on:
      - redis
  redis:
    image: redis:7-alpine
    ports:
      - "6379:6379"
  tests:
    build: .
    command: pytest tests/ -v --cov=src
    depends_on:
      - api
```

5.3.3 Docker Images and Containers

Table 5.13: Docker Images for Sanskrit NLP Project

Image Name	Purpose	Size

sanskrit-nlp-api:latest	Production API server	1.2 GB
sanskrit-nlp-api:dev	Development with hot-reload	1.4 GB
sanskrit-nlp-tests:latest	Test runner with pytest	1.2 GB
sanskrit-nlp-batch:latest	Batch processing for large corpora	1.3 GB

5.3.4 CI/CD Pipeline with Docker and GitHub Actions

Table 5.14: CI/CD Pipeline Stages

Stage	**Docker Command**	**Trigger**
Build	docker build -t sanskrit-nlp-api:$TAG .	On push to develop
Test	docker run sanskrit-nlp-tests:latest	On PR to main
Push	docker push ghcr.io/sanskrit-nlp/api:$TAG	On merge to main
Deploy (Staging)	docker compose -f docker-compose.staging.yml up -d	On merge to develop
Deploy (Prod)	docker compose -f docker-compose.prod.yml up -d	On tag release/v*

5.3.5 Docker Hub / GitHub Container Registry Strategy

- **Registry**: GitHub Container Registry (GHCR) – integrated with CI/CD
- **Image naming**: `ghcr.io/sanskrit-nlp/sanskrit-nlp-api:latest`
- **Tagging strategy**:
  - `latest` – most recent build from develop branch
  - `v1.0.0` – versioned release tags
  - `sha-xxxxx` – commit hash for traceability
- **Retention**: Keep last 10 images per repository

5.4 Experiment 4: XP Programming – Process Methodology, Framework and Limitations

5.4.1 Introduction to Extreme Programming (XP)

Extreme Programming (XP) is an Agile software development methodology that aims to produce higher quality software and higher quality of life for the development team [14]. XP emphasizes technical excellence and customer satisfaction.

5.4.2 XP Process Methodology

Table 5.15: XP Practices for Sanskrit NLP Project

XP Practice	Implementation	Benefit
Planning Game	User stories with acceptance criteria	Clear requirements
Small Releases	Weekly API updates	Fast feedback
Metaphor	”Panini grammar as rule engine”	Shared understanding
Simple Design	Minimal code, YAGNI principle	Reduced complexity
Testing (TDD)	Write tests before code	High confidence
Refactoring	Monthly code cleanup	Maintainable code
Pair Programming	Two engineers + linguist for complex rules	40% error reduction
Collective Ownership	Any engineer can fix any bug	No silos
Continuous Integration	GitHub Actions on every commit	Early defect detection
Sustainable Pace	40-hour weeks	Consistent velocity
On-site Customer	Linguist full-time	Immediate clarifications
Coding Standards	PEP 8 + extensions	Consistent style

5.4.3 XP Framework

5.4.4 XP Framework Limitations

Table 5.16: XP Limitations and Mitigation Strategies

Limitation	Mitigation for Sanskrit NLP Project
Requires on-site customer	Sanskrit linguist available full-time; budget allocated
Pair programming is resource intensive	Used only for complex Panini rules; not for all tasks
Difficult for large teams	Team size limited to 6 members
Requires cultural shift	Gradual adoption over 3 sprints with coaching
Documentation can suffer	Dedicated documentation sprint every 4 sprints
Not suitable for fixed-price contracts	Hybrid approach: fixed scope for core Panini rules
Scales poorly across multiple teams	Single team for entire project

5.4.5 XP vs Scrum vs Lean Comparison

Table 5.17: Comparison of Agile Methodologies Used

Aspect	Scrum	XP	Lean
Primary focus	Project management	Engineering practices	Waste elimination
Roles	Product Owner, Scrum Master, Team	No explicit roles	No explicit roles
Ceremonies	Sprint Planning, Review, Retro, Daily Scrum	Planning Game, Stand-up	Continuous improvement
Key practices	Backlog, Burndown, Timeboxing	TDD, Pair programming, CI	Value stream mapping
Artifacts	Product Backlog, Sprint Backlog, Increment	User stories, Tests, Code	Kanban board

Best for	Complex product development	Technical excellence
Process optimization		

5.4.6 Hybrid Approach for Sanskrit NLP Project

The project uses a hybrid approach combining:

- **Scrum** for project management (sprints, roles, ceremonies)
- **XP** for engineering practices (TDD, pair programming, CI)
- **Lean** for waste elimination (automated documentation, value stream mapping)

5.4.7 Process Methodology Summary

Table 5.18: Process Methodology Summary

Phase	**Methodology**	**Key Activities**
Requirements	Scrum	Product backlog, user stories, prioritization
Development	XP	TDD, pair programming, refactoring
Integration	XP + DevOps	CI/CD, ten-minute build
Review	Scrum	Sprint review, stakeholder feedback
Improvement	Lean	Retrospective, waste elimination

5.4.8 Continuous Integration Pipeline (Complete)

```
Push to any branch
       |
       v
+--------------+
|    Lint      | (flake8, black --check, isort)
+--------------+
       |
       v
+--------------+
```

```
+------------------+
|   User Stories   |
+------------------+
         |
         v
+------------------+
|   Release Plan   |
+------------------+
         |
         v
+------------------+     +------------------+
|    Iteration     |---->|   Acceptance     |
|    Planning      |     |    Tests         |
+------------------+     +------------------+
         |
         v
+------------------+
|   Pair Coding    |
+------------------+
         |
         v
+------------------+     +------------------+
|    Unit Tests    |---->|   Integration    |
|    (TDD)         |     |    (CI)          |
+------------------+     +------------------+
         |
         v
+------------------+
|   Small Release  |
+------------------+
```

Figure 5.1: XP Process Flow for Sanskrit NLP Project

```
|  Type Check  | (mypy --strict)
+--------------+
       |
       v
+--------------+
|  Unit Tests  | (pytest --cov=src --cov-fail-under=85)
+--------------+
       |
       v
+--------------+
| Integration  | (pytest tests/integration)
+--------------+
```

```
        |
        v
+--------------+
| Build Docker | (docker build -t sanskrit-nlp-api .)
+--------------+
        |
        v
+--------------+
| Deploy Staging| (if branch = develop or main)
+--------------+
```

5.4.9 Weekly Cycle Effectiveness Metrics

Table 5.19: Weekly Cycle Effectiveness Over Time

Metric	**Week 1**	**Week 4**	**Week 8**
Story points completed	18	24	26
Defects discovered	12	7	4
Code review turnaround	8 hours	4 hours	2 hours
Pair programming sessions	2	4	6
Linguist feedback time	24 hours	12 hours	4 hours

5.4.10 Pair Programming Benefits Quantified

Table 5.20: Pair Programming Benefits

Benefit	**Before Pair Programming**	**After Pair Programming**
Rule implementation errors	15%	6%
Code review defects	8 per sprint	3 per sprint
Knowledge transfer time	2 weeks	3 days
Bug fix time (critical)	8 hours	3 hours

5.4.11 Conclusion for Unit 5

The Scrum framework successfully structured the Sanskrit NLP project with defined roles (Product Owner, Scrum Master, Development Team), artifacts (Product Backlog, Sprint Back-

log, Increment), and ceremonies (Sprint Planning, Daily Scrum, Sprint Review, Retrospective) [15]. Best practices including Definition of Done, time-boxed events, and sustainable pace were consistently followed. The ten-minute build was achieved at 7.75 minutes, meeting the XP standard. Continuous Integration via GitHub Actions provided rapid feedback on code quality. Docker containerization ensured consistent deployment across environments, and the CI/CD pipeline automated testing and deployment [14]. XP practices, particularly pair programming and TDD, reduced implementation errors by 60%. The hybrid Agile approach successfully balanced the need for rigorous Panini grammar accuracy with flexible, iterative development, delivering working software every two weeks while maintaining linguistic precision.

Table 5.21: Final Project Summary

Parameter	**Value**
Total Project Duration (Baseline)	218 days
Total Project Duration (Crashed)	200 days
Project Budget	INR 33–41 Lakhs
NPV (10% discount)	INR +37.72 Lakhs
Payback Period	2.14 years
Critical Path	A $\rightarrow$ C $\rightarrow$ D $\rightarrow$ E $\rightarrow$ G $\rightarrow$ H $\rightarrow$ I
Team Size	10 members
Sprint Duration	2 weeks
Average Velocity	26 story points per sprint
Code Coverage	85%
Unit Test Pass Rate	87.6% $\rightarrow$ 95% (target)

Bibliography

[1] R. E. Freeman, "The stakeholder approach revisited," *Zeitschrift für Wirtschafts- und Unternehmensethik*, vol. 5, no. 3, pp. 228–241, 2004.

[2] P. M. Institute, *A Guide to the Project Management Body of Knowledge (PMBOK Guide)*, 7th ed. Newtown Square, PA: Project Management Institute, 2021.

[3] R. S. Pressman, *Software Engineering: A Practitioner's Approach*, 8th ed. New York, NY: McGraw-Hill, 2014.

[4] Panini, *Ashtadhyayi*. Traditional Sanskrit Grammar Text, 500 BCE, classical Sanskrit grammar text, c. 500 BCE.

[5] INRIA, "Sanskrit heritage engine," https://sanskrit.inria.fr, 2025.

[6] DSDM Consortium, "Moscow method for prioritization," Agile Business Consortium, 2014, available at: https://www.agilebusiness.org.

[7] P. Goyal and G. Huet, "Design and analysis of a sanskrit morphological analyzer," *ACM Transactions on Asian and Low-Resource Language Information Processing*, vol. 15, no. 3, pp. 1–22, 2016.

[8] H. Kerzner, *Project Management: A Systems Approach to Planning, Scheduling, and Controlling*, 12th ed. Hoboken, NJ: Wiley, 2017.

[9] A. Kulkarni and D. Pawar, "Design of a sanskrit morphological analyzer," in *Proceedings of the 5th International Conference on Natural Language Processing*, Kharagpur, India, 2010, pp. 89–96.

[10] B. W. Boehm, *Software Engineering Economics*. Englewood Cliffs, NJ: Prentice-Hall, 1981.

[11] S. R. Clegg, T. Skyttermoen, and A. L. Vaagaasar, "Project management: A stakeholder approach," *Project Management Journal*, vol. 41, no. 4, pp. 5–14, 2010.

[12] Government of India, "National mission for manuscripts," https://www.namami.gov.in, 2025.

[13] University of Cambridge, "Digital corpus of sanskrit (dcs)," https://www.sanskrit-linguistics.org/dcs, 2025.

[14] K. Beck, *Extreme Programming Explained: Embrace Change*. Boston, MA: Addison-Wesley, 2000.

[15] K. Schwaber and J. Sutherland, *The Scrum Guide*. Scrum.org, 2020, available at: https://scrumguides.org.

Appendices

Appendix A: Detailed Work Breakdown Structure (WBS)

Table 5.22: Detailed Work Breakdown Structure

WBS Code	Task	Effort (days)	Responsible
1.0	Project Management	30	PM
1.1	Requirement gathering	14	PM + Linguist
1.2	Stakeholder communication	Ongoing	PM
1.3	Status reporting	Ongoing	PM
1.4	Risk management	Ongoing	PM
2.0	Data Collection	40	Data Scientist
2.1	Corpus identification	10	Linguist
2.2	Digitization of manuscripts	15	Data Scientist
2.3	Expert annotation	38	Linguist
2.4	Gold standard creation	10	Linguist + QA
3.0	Panini Rule Engine	53	NLP Engineer 1
3.1	Sutra extraction (Ashtadhyayi)	20	Linguist
3.2	Rule formalization	25	NLP Engineer 1
3.3	Rule validation	8	Linguist
4.0	Morphological Analyzer	46	NLP Engineer 2
4.1	Sandhi splitting module	15	NLP Engineer 2
4.2	Stemming algorithm	12	NLP Engineer 2
4.3	Tagging system	10	NLP Engineer 2
4.4	Accuracy optimization	9	NLP Engineer 2 + Linguist
5.0	Semantic Parser	41	NLP Engineer 3
5.1	Dependency parsing	15	NLP Engineer 3
5.2	Semantic role labeling	14	NLP Engineer 3
5.3	Meaning representation	12	NLP Engineer 3
6.0	Knowledge Extraction	29	NLP Engineer 1
6.1	Triple generation	10	NLP Engineer 1

6.2	Knowledge graph construction	12	NLP Engineer 1
6.3	Query interface	7	NLP Engineer 1
7.0	API Development	21	NLP Engineer 2
7.1	REST endpoint design	5	NLP Engineer 2
7.2	Authentication	4	NLP Engineer 2
7.3	Documentation (Swagger)	6	NLP Engineer 2
7.4	Rate limiting	6	NLP Engineer 2
8.0	Testing & QA	29	QA Lead
8.1	Unit testing	10	QA Lead
8.2	Integration testing	8	QA Lead
8.3	Performance testing	6	QA Lead
8.4	Accuracy validation	5	Linguist
9.0	Deployment & Closure	14	All
9.1	Cloud deployment	5	DevOps
9.2	User training	4	PM + Linguist
9.3	Documentation finalization	3	All
9.4	Project closure report	2	PM

Appendix B: Detailed Risk Register

Table 5.23: Complete Risk Register

ID	Risk Description	Prob	Impact	Score	Mitigation Status
R01	Shortage of Sanskrit linguists with NLP knowledge	0.7	0.8	0.56	Active
R02	Panini rule formalization takes longer than expected	0.5	0.8	0.40	Active
R03	Low accuracy on complex sandhi phenomena	0.5	0.7	0.35	Active
R04	Cloud cost overrun due to large corpus processing	0.6	0.5	0.30	Active
R05	Low adoption by academic institutions	0.3	0.7	0.21	Monitoring

R06	Dependency on third-party Sanskrit corpus licenses	0.4	0.5	0.20	Contingency
R07	API security vulnerability	0.3	0.6	0.18	Monitoring
R08	Team member turnover	0.3	0.5	0.15	Monitoring
R09	Performance degradation at scale	0.4	0.4	0.16	Active
R10	Incompatibility with existing digital libraries	0.2	0.4	0.08	Low

Appendix C: Complete Test Cases

C.1 Noun Declension Tests

Table 5.24: Noun Declension Test Cases

Word	**Case**	**Expected Stem**	**Actual Stem**	**Status**
raama.h	Nominative Singular	raama	raam	FAIL
raamam	Accusative Singular	raama	raama	PASS
raamena	Instrumental Singular	raama	raama	PASS
raamaaya	Dative Singular	raama	raama	PASS
raamaat	Ablative Singular	raama	raama	PASS
raamasya	Genitive Singular	raama	raama	PASS
raame	Locative Singular	raama	raama	PASS
raamaah	Nominative Plural	raama	raama	PASS

C.2 Verb Conjugation Tests

Table 5.25: Verb Conjugation Test Cases

Verb Form	**Root Expected**	**Root Actual**	**Tense**	**Status**
gacchati	gam	gacch	Present	FAIL
gacchanti	gam	gacch	Present	FAIL
agacchat	gam	gacch	Past	FAIL

gamiSyati	gam	gam	Future	PASS
bhavati	bhuu	bhuu	Present	PASS
karoti	kR	kar	Present	FAIL

C.3 Sandhi Splitting Tests

Table 5.26: Sandhi Splitting Test Cases

Compound Word	Expected Split	Actual Split	Status
devaalaya.h	deva + aalaya	deva + aalaya	PASS
raama.h + uvacha	raama + uvaacha	raama + uvaacha	PASS
hari.h + iisha.h	hari + iisha	hari + iisha	PASS
suryodaya.h	surya + udaya	surya + udaya	PASS

C.4 Test Execution Log (Sprint 4)

Table 5.27: Test Execution Log

Date	Test Suite	Passed	Failed	Comments
2026-05-15	Morphological Analyzer	38	7	2 critical failures
2026-05-16	Semantic Parser	21	4	Root extraction issue
2026-05-17	Knowledge Extraction	13	2	Triple format correct
2026-05-18	API Endpoints	12	0	All passing
2026-05-19	Integration	8	0	All passing

Appendix D: Sprint Burndown Data (All Sprints)

Table 5.28: Sprint Performance Summary

Sprint	Planned	Completed	Velocity	Defects	Retro Item
1	25	18	18	12	Improve testing
2	28	22	22	9	Add pair programming

3	30	24	24	7	Automate validation
4	30	26	26	4	Split large stories
5	30	28	28	3	Continue current
6	32	30	30	2	Document patterns

D.1 Sprint Burndown – Detailed Data

Table 5.29: Sprint Burndown Detailed Data

Day	Sprint 1	Sprint 2	Sprint 3	Sprint 4	Sprint 5
0	25	28	30	30	30
2	22	25	27	28	27
4	18	22	24	25	24
6	14	18	20	20	19
8	9	12	14	14	12
10	7	6	6	4	2

Appendix E: Project Charter

Table 5.30: Project Charter

Element	Details
Project Title	AI-Powered Sanskrit NLP Framework for Morphological and Semantic Analysis
Project Start Date	April 1, 2026
Project End Date	November 30, 2026 (218 days)
Project Budget	INR 33–41 Lakhs
Project Manager	[Name]
Project Sponsor	UGC – Digital Heritage Initiative
Business Case	Preservation of Paninian grammar through AI, enabling digital Sanskrit research
Project Objectives	1. Develop morphological analyzer with 500+ Panini sutras 2. Achieve 92%+ accuracy on classical Sanskrit texts 3. Deploy accessible API for researchers

Success Criteria	1. Accuracy $\geq 92\%$ 2. API response < 2 seconds 3. Adoption by 15+ institutions by Year 2
Key Stakeholders	UGC, IGNCA, SRM University, IIT (BHU)

Appendix F: Stakeholder Communication Plan

Table 5.31: Stakeholder Communication Plan

Stakeholder	**Communication Method**	**Frequency**	**Format**
UGC / MHRD	Formal status report	Monthly	PDF (1-2 pages)
Sanskrit Linguists	Joint review session	Weekly	Meeting + notes
NLP Engineers	Daily stand-up	Daily	Verbal + Slack
Researchers	Newsletter	Monthly	Email
IGNCA	Quarterly review	Quarterly	Presentation
Graduate Students	Documentation	Per release	Wiki
Steering Committee	Steering meeting	Bi-monthly	Presentation

Appendix G: Quality Assurance Checklist

Table 5.32: Quality Assurance Checklist

ID	**Check Item**	**Standard**	**Frequency**	**Responsible**
QA01	Code linting	flake8 zero warnings	Per commit	Engineer
QA02	Unit test coverage	$\geq 85\%$	Per sprint	QA Lead
QA03	Documentation update	API docs current	Per release	Engineer
QA04	Linguist validation	92% accuracy	Weekly	Linguist
QA05	Performance test	< 2 sec/verse	Per release	QA Lead
QA06	Security scan	No high severity	Monthly	DevOps
QA07	Code duplication	$< 5\%$	Monthly	Engineer
QA08	Type checking	mypy strict	Per commit	Engineer

Appendix H: Tools and Technologies Used

Table 5.33: Tools and Technologies

Category	Tool	Purpose
Version Control	Git / GitHub	Source code management, collaboration
CI/CD	GitHub Actions	Automated testing and deployment
Containerization	Docker	Environment consistency
Programming	Python 3.10	Primary development language
NLP	spaCy, PyTorch	Morphological analysis, ML models
API	FastAPI	REST API development
Testing	pytest	Unit and integration testing
Documentation	Sphinx, Swagger	API and user documentation
Project Management	JIRA, Trello	Sprint tracking, backlog management
Communication	Slack, Teams	Daily stand-ups, team communication
Cloud	AWS/GCP	Infrastructure hosting

Appendix I: Lessons Learned

What Went Well

1. Pair programming between NLP engineers and linguists reduced rule errors by 40%
2. Weekly small releases enabled fast feedback from stakeholders
3. Docker containerization eliminated environment inconsistencies
4. Daily stand-ups improved team coordination and blocker resolution

What Could Be Improved

1. Initial test coverage was low (72%) – should start with 85% target
2. Story sizes should be capped at 5 points for better predictability
3. Linguist involvement should begin earlier in the development cycle
4. Documentation should be updated continuously, not at sprint end

Recommendations for Future Projects

1. Invest in automated sandhi validation from day one
2. Establish gold-standard test dataset before development begins
3. Schedule regular knowledge transfer sessions between domain experts and engineers
4. Use feature flags to enable incremental deployment of Panini rules

Appendix J: Acronyms and Glossary

Table 5.34: Glossary of Terms

Term	Definition
Ashtadhyayi	Panini's classical Sanskrit grammar text with approximately 4,000 sutras
Sandhi	Phonetic merging of sounds at word boundaries in Sanskrit
Sutra	An aphoristic rule or formula in Paninian grammar
Subanta	Nominal inflection in Sanskrit grammar
Tinanta	Verbal inflection in Sanskrit grammar
IAST	International Alphabet of Sanskrit Transliteration – standard Romanization scheme
KLOC	Thousand Lines of Code – software size metric
Function Point	Unit of measurement for software functionality
Earned Value	Measure of work performed expressed in terms of budget
Velocity	Amount of work completed per sprint in story points
Technical Debt	Implied cost of additional rework caused by choosing easy solution now

Appendix K: Budget Breakdown by Phase

Table 5.35: Budget Breakdown by Phase (INR Lakhs)

Phase	Development	Data	Infra	Total
Initiation	2	1	0.5	3.5
Requirements	3	2	0.5	5.5
Design	4	1	0.5	5.5
Development	8	2	1	11
Testing	2	1	0.5	3.5
Deployment	1	0	1	2
Maintenance (Year 1)	2	1	1	4
Total	**22**	**8**	**5**	**35**

Appendix L: Resource Loading Chart

Table 5.36: Resource Loading by Month

Month	**PM**	**Linguists**	**NLP Eng**	**Data Sci**	**QA**
Month 1	100%	50%	25%	50%	0%
Month 2	100%	100%	50%	100%	0%
Month 3	100%	100%	100%	50%	25%
Month 4	100%	75%	100%	25%	50%
Month 5	100%	50%	100%	0%	100%
Month 6	100%	25%	75%	0%	100%
Month 7	50%	0%	25%	0%	50%